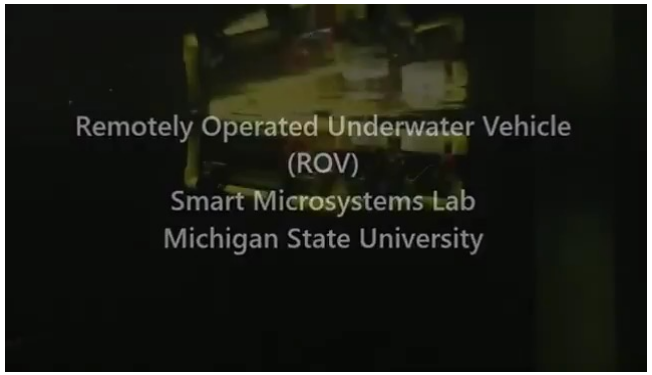
Politecnico di Torino

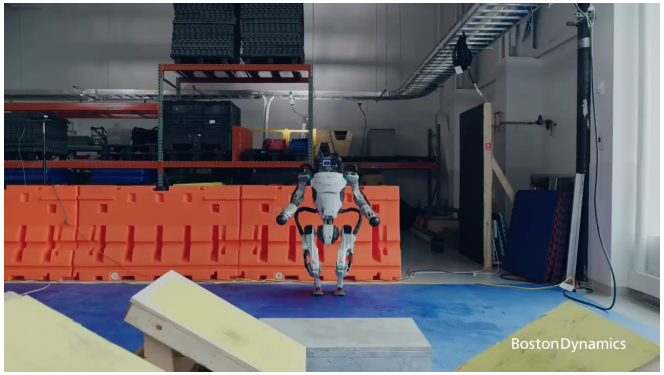
VANDAL

Robot Learning

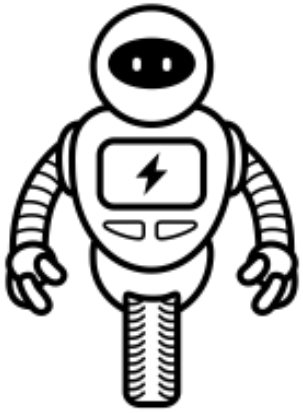
Lecture 2c. Mobile Robots

Mobile robots





Wheeled Robots



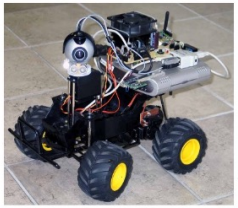
Wheeled mobile robots

2- and 4-wheel
Differential driving

Tricyle

Murata Boy and Girl
Bicycle and Unicycle

Boston Dynamics Handle



Car-like
Ackermann

6-wheel
space rovers

4-wheel
steering

Tri-bots
Omniwheels

Wheeled mobile robots

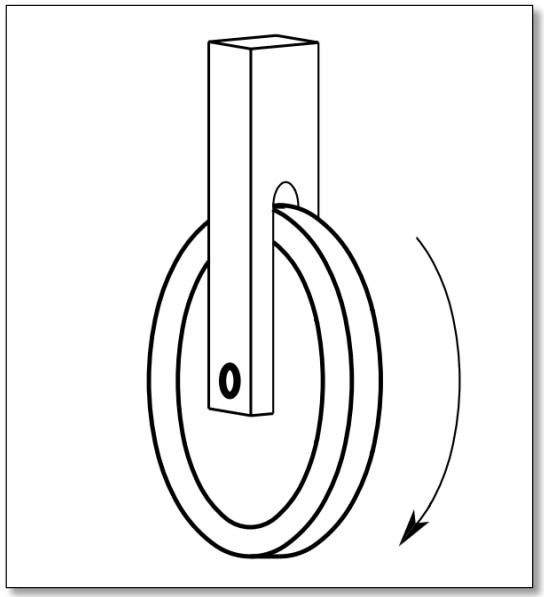
There are many kinds of mobile robots that use wheels for moving.

Their **maneuverability** depends on:

- **Type of wheels**: their mechanical structure, actuation
- **Geometry of wheels**: the relative placement of the wheels on the chassis

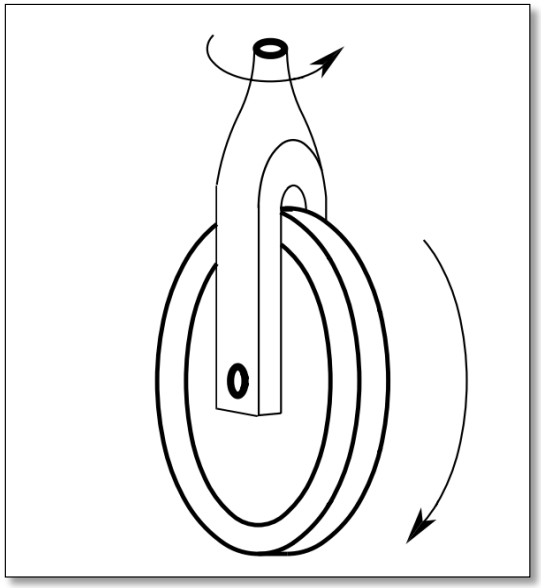
Type of Wheels

Fixed wheel



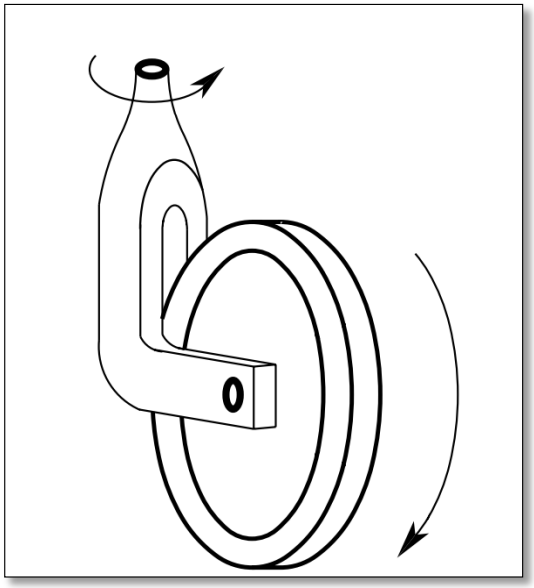
- Fixed orientation w.r.t. chassis
- Can be active or passive

Steerable wheel



- Variable orientation w.r.t. chassis
- Typically, it is active

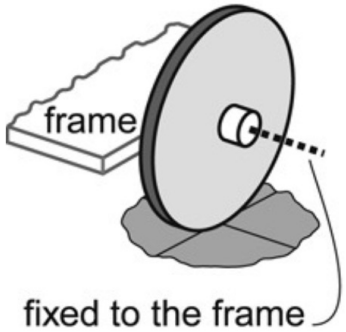
Castor wheel

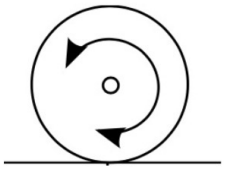


- Variable orientation w.r.t. chassis
- Typically, it is passive
- It automatically aligns with the direction of motion

Type of Wheels

fixed wheel

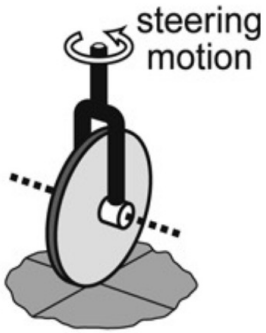


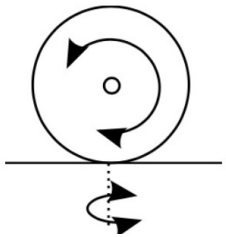


1 DoF:

- rotation around the axle

steering wheel

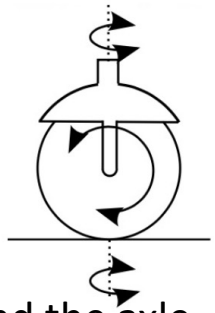




2 DoF:

- rotation around the axle
- rotation around the contact point with the ground

castor wheel

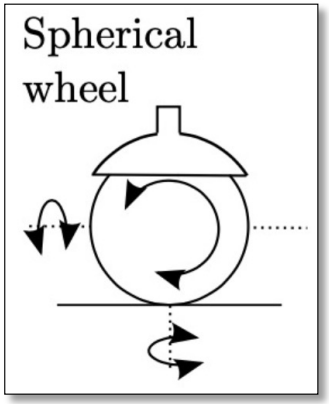


3 DoF:

- rotation around the axle
- rotation around the contact point with the ground
- rotation around the caster axis

Type of Wheels

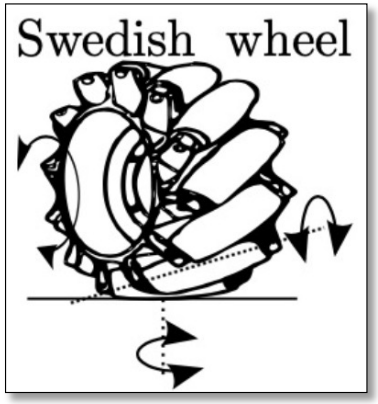
Spherical Wheels



3 DoF:

- Rotation in any direction
- Rotation around the contact point with the ground

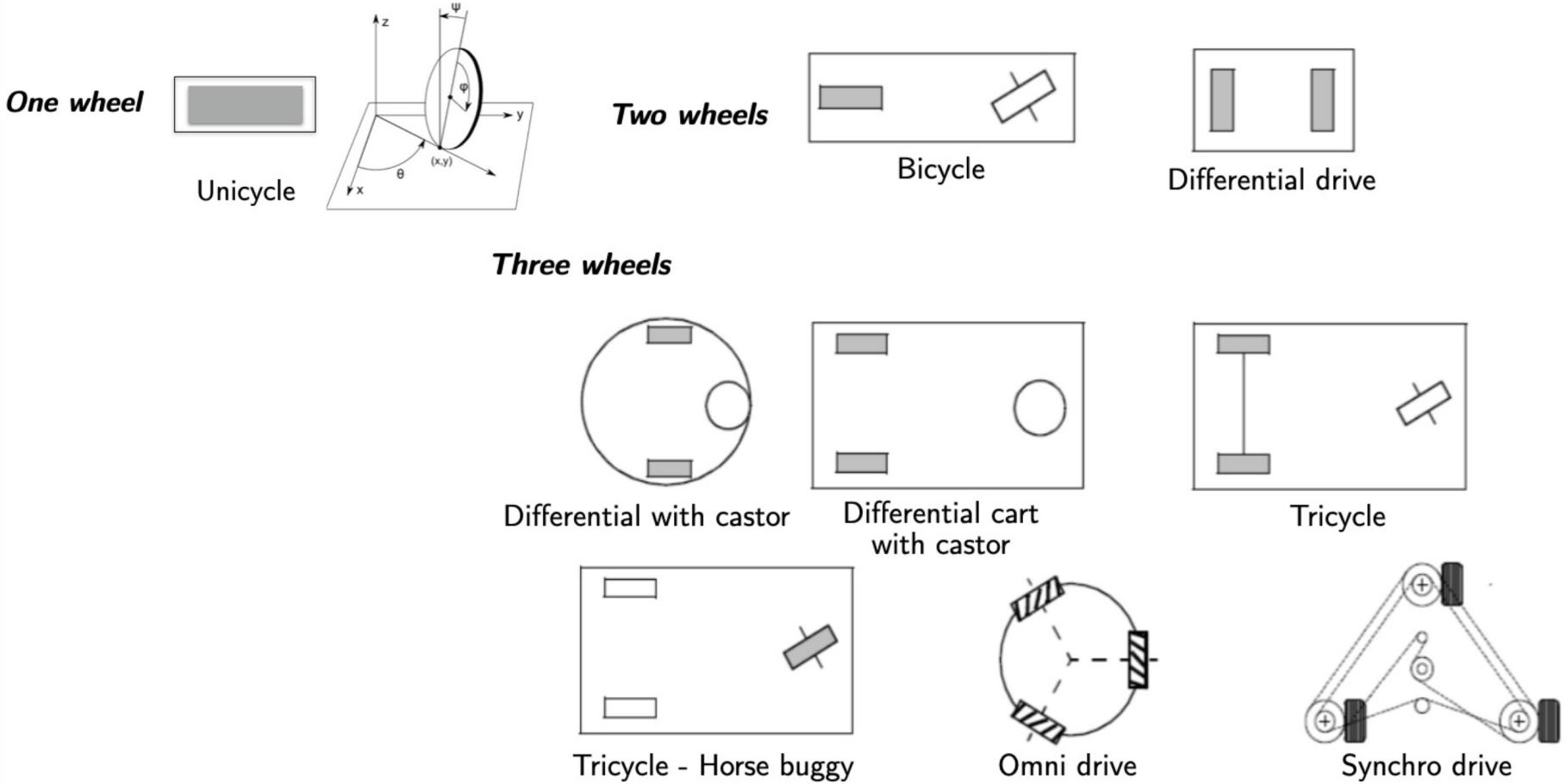
Omniwheels (Mecanum wheels, Swedish wheels)



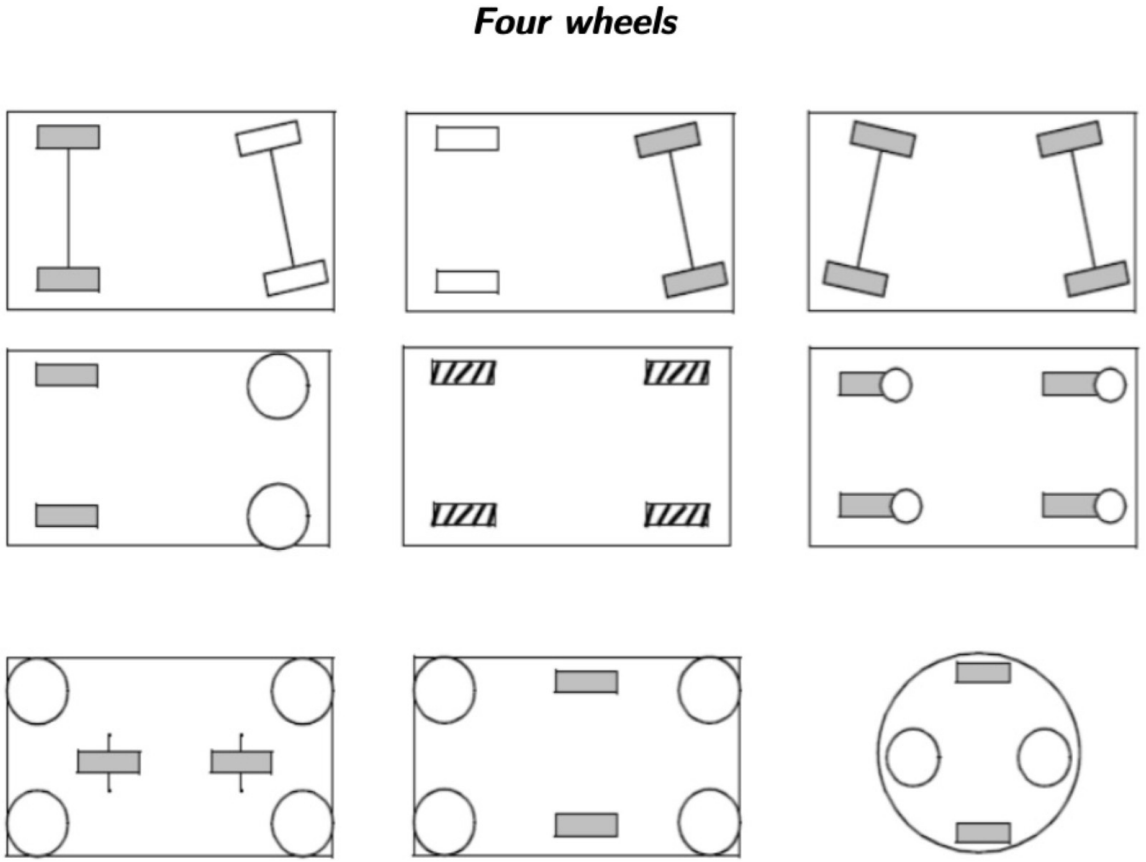
3 DoF:

- Rotation around the wheel axle
- Rotation around the contact point with the ground
- Rotation around the roller axle

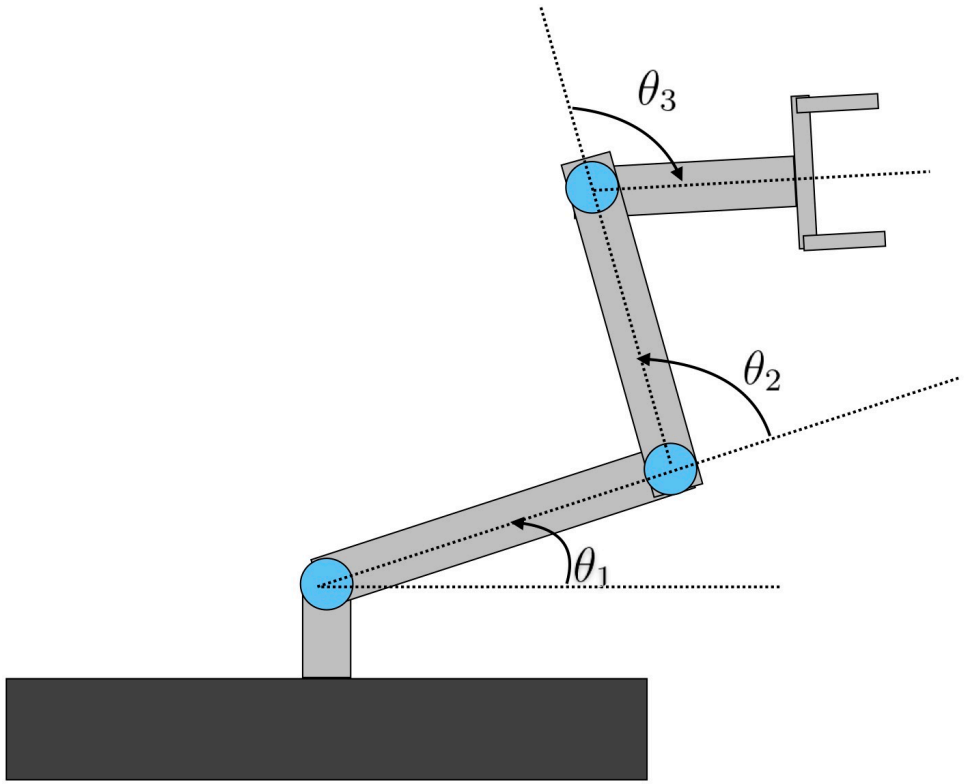
Geometry of wheeled robots



Geometry of wheeled robots



Non-holonomic constraints



- The robot starts with a certain joint configuration

$$\boldsymbol{\theta}(t_o)$$

- A certain motion profile is applied to the joints

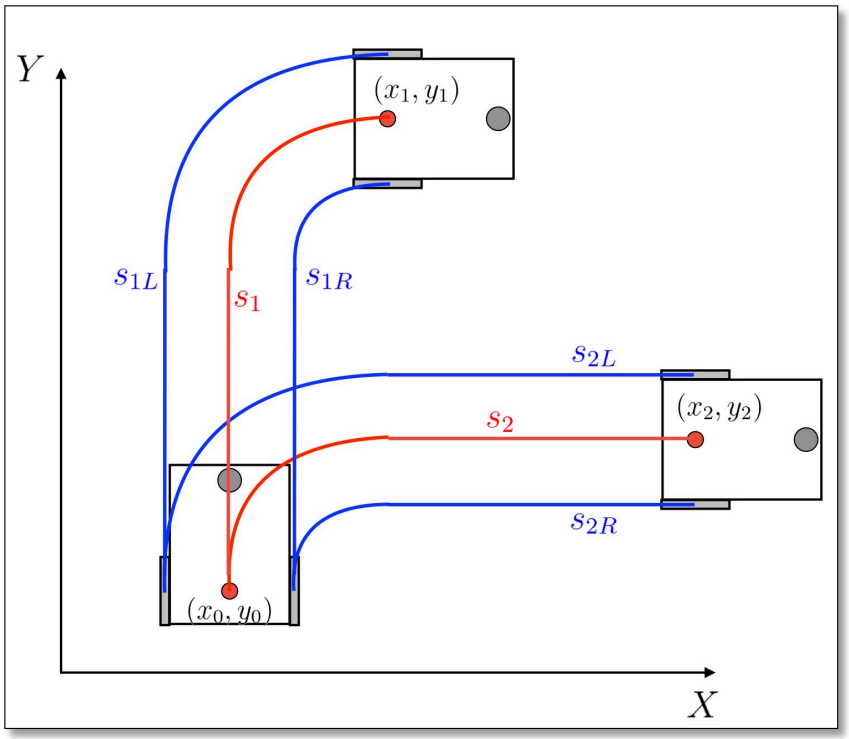
$$\dot{\boldsymbol{\theta}}(t)$$

- If we integrate the joint velocities, we can univoquely compute the pose of the end-effector as a function of the joint variables

$$\boldsymbol{x}(t_f) = f\Big(\boldsymbol{\theta}(t_f)\Big)$$

Non-holonomic constraints

- **Configuration variables** are the rotation angles of the wheels: $\boldsymbol{q} = [\phi_l, \phi_r]$
- Radius of the wheels: r



The **traveled distance** by each wheel and by the robot are:

$$s_{1l} = \phi_{1l} r \qquad s_{1r} = \phi_{1r} r \quad \Rightarrow \quad s_1 = \frac{s_{1l} + s_{1r}}{2}$$

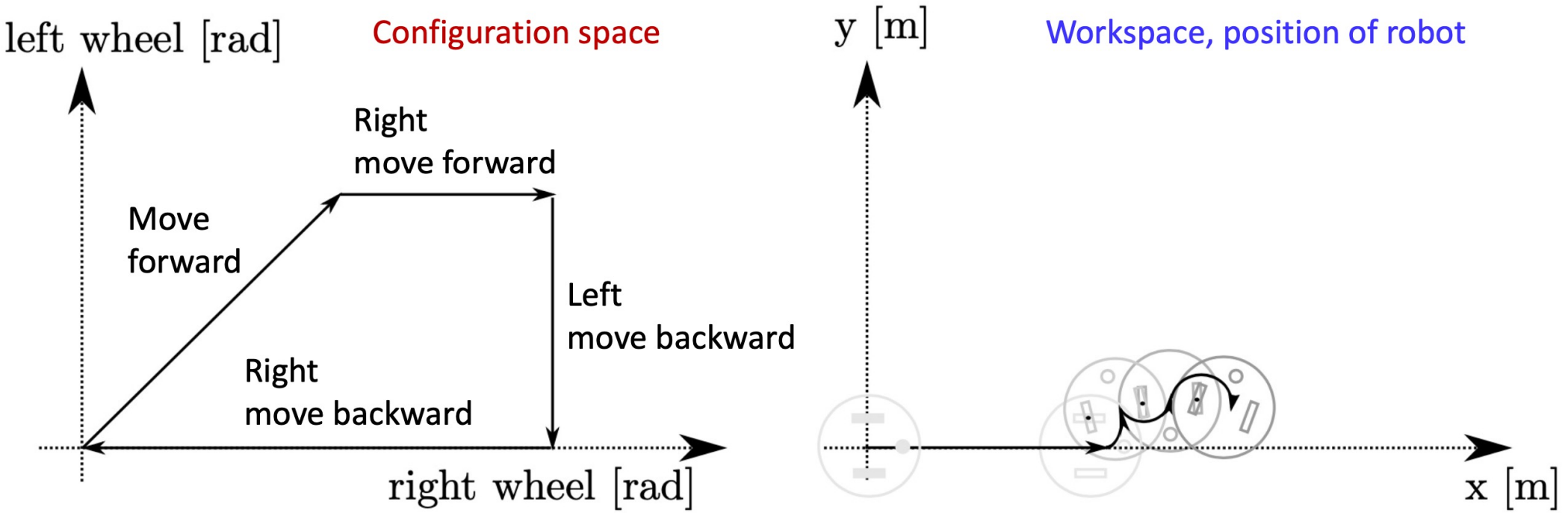
Despite traveling the same distance, the robots reach different positions

$$s_{1l} = s_{2l} \qquad s_{1r} = s_{2r} \quad \Rightarrow \quad s_1 = s_2 \text{ but } (x_1, y_1) \neq (x_2, y_2)$$

To determine the configuration of the robot in task space it is necessary to consider how the motion was executed

=> **Differential kinematics**

Non-holonomic constraints

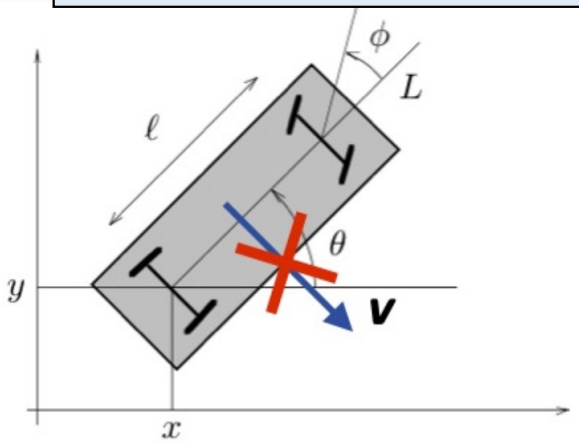


Closed trajectories in the configuration space do not necessarily correspond to closed trajectories in the task space!

Non-holonomic constraints

A **kinematic constraint** imposes restrictions on the **achievable velocities** of the robot.
It is based on a functional relation among configuration variables q and their derivatives, $\dot{q}$

$$f(\boldsymbol{q}, \dot{\boldsymbol{q}}) = 0$$



A kinematic constraint is called **holonomic** if it can be integrated to obtain a relationship solely between the configuration variables q

$$f(\boldsymbol{q}, \dot{\boldsymbol{q}}) = 0 \Rightarrow g(\boldsymbol{q}) = 0$$

A **non-holonomic constraint** is a kinematic constraint that **cannot be integrated to remove the derivatives of the configuration variables**. A non-holonomic constraint **does not limit the accessible task configurations** but **limits the trajectories to reach them** (i.e. the instantaneous motions)

Pure-rolling

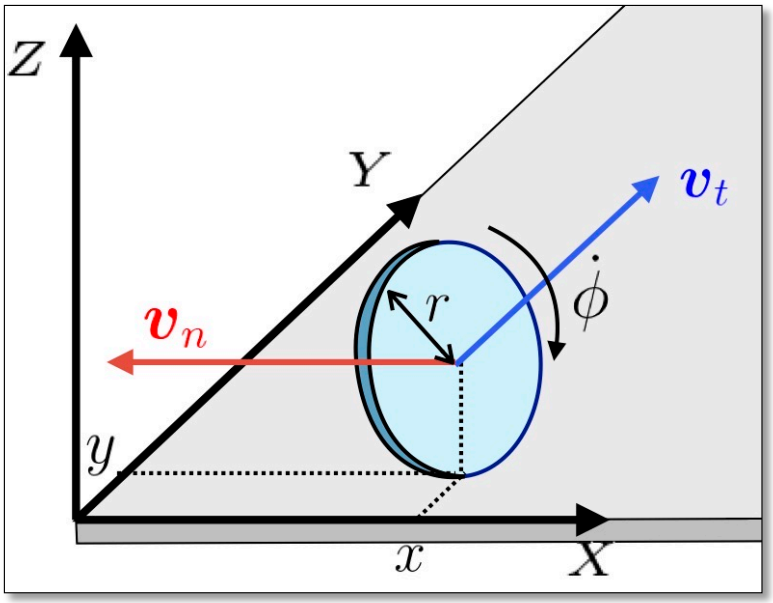
Each standard wheel (in normal working conditions) introduces two constraints

- **No slipping constraint**: the velocity of the center of the wheel is parallel to the wheel plane

$$|\boldsymbol{v}_n| = 0$$

- **No skidding condition**: the magnitude of the velocity of the center of the wheel is proportional to the wheel velocity

$$|\boldsymbol{v}_t| = r\,\dot{\phi}$$

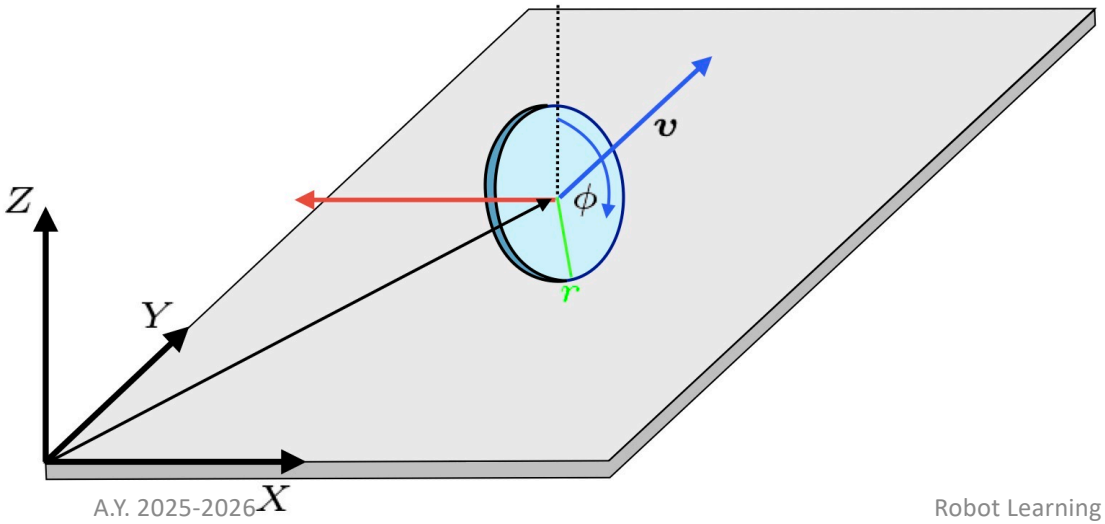


The **no-slipping constraint** is a **non-holonomic constraint**, because it does not allow motion in the direction that is normal to the rolling direction

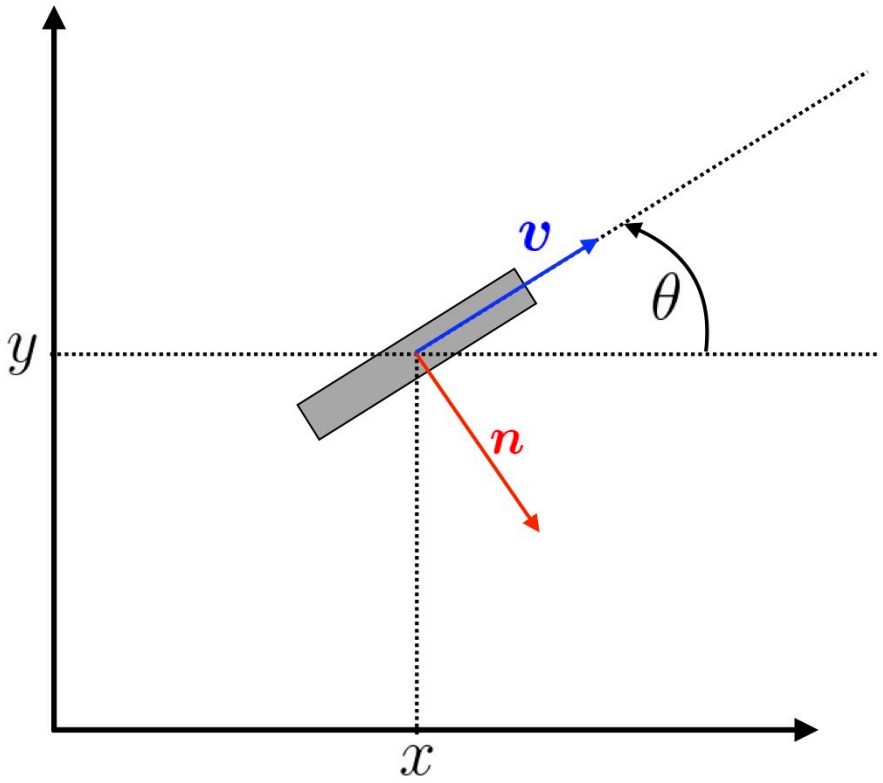
Unicycle

- The simplest wheeled robot is the unicycle. It is an abstract robot that is modeled as a **single steerable wheel**, that rolls without slipping.

- The wheel is assumed to be stable upright...



Unicycle



- The generalized coordinates of the unicycle are:

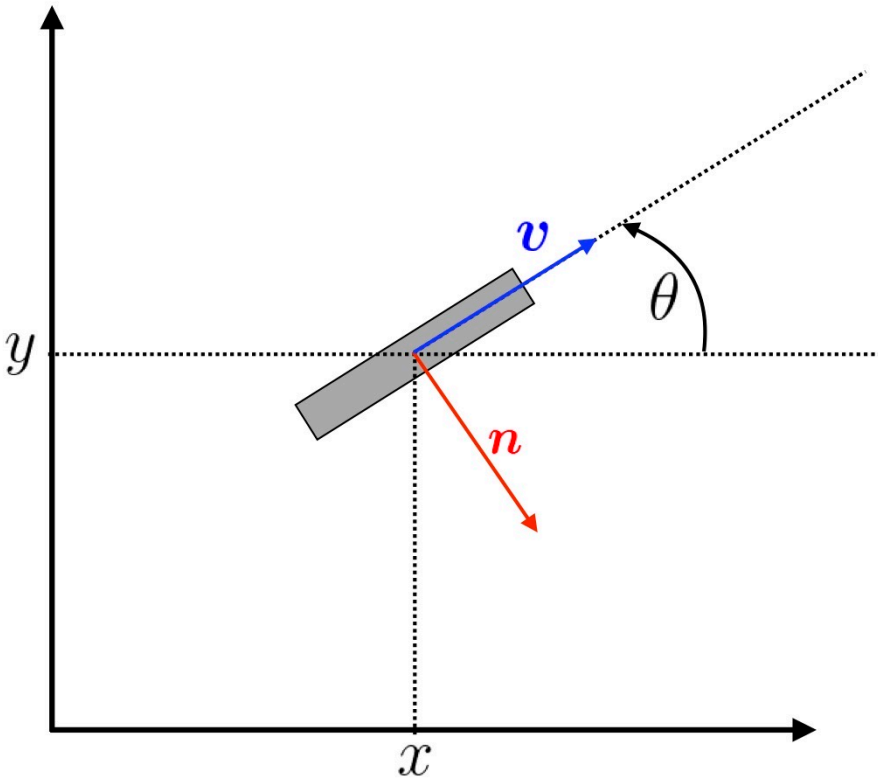
$$\boldsymbol{q} = [x,\ y,\ \theta]^T$$

- $\boldsymbol{n}$ is the orthogonal direction to the wheel

- The **pure-rolling constraint** is expressed as

$$\begin{pmatrix} \sin(\theta) & -\cos(\theta) \end{pmatrix} \begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = \boldsymbol{n}^T \begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = 0$$

Unicycle



- We can rewrite the constraint so that all coordinates appear

$$\begin{pmatrix} \sin(\theta) & -\cos(\theta) & 0 \end{pmatrix} \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} = A(\boldsymbol{q})\, \dot{\boldsymbol{q}}$$

A constraint in this form is called **Pfaffian**

- The admissible velocities $\dot{\boldsymbol{q}}$ must belong to the null space of $A^T(\boldsymbol{q})$

Unicycle

- No-slipping constraint: $\begin{pmatrix} \sin(\theta) & -\cos(\theta) & 0 \end{pmatrix} \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} = A(\boldsymbol{q})\ \dot{\boldsymbol{q}}$

- The admissible velocities $\dot{\boldsymbol{q}}$ must belong to the null space of $A^T(\boldsymbol{q})$

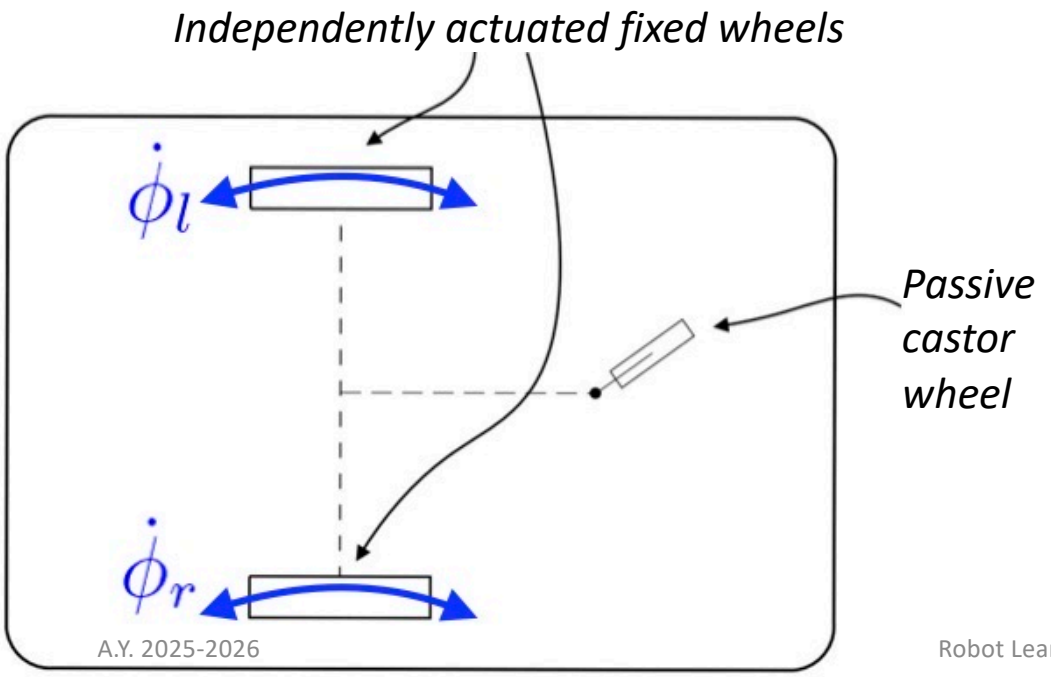
$$\dot{\boldsymbol{q}} = \begin{pmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{pmatrix} = v \begin{pmatrix} \cos(\theta) \\ \sin(\theta) \\ 0 \end{pmatrix} + \omega \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} v\cos(\theta) \\ v\sin(\theta) \\ \omega \end{pmatrix}$$

Bases of the null space of $A^T(\boldsymbol{q})$

- This kinematic model describes all the admissible instantaneous motions
- v and ω are the inputs

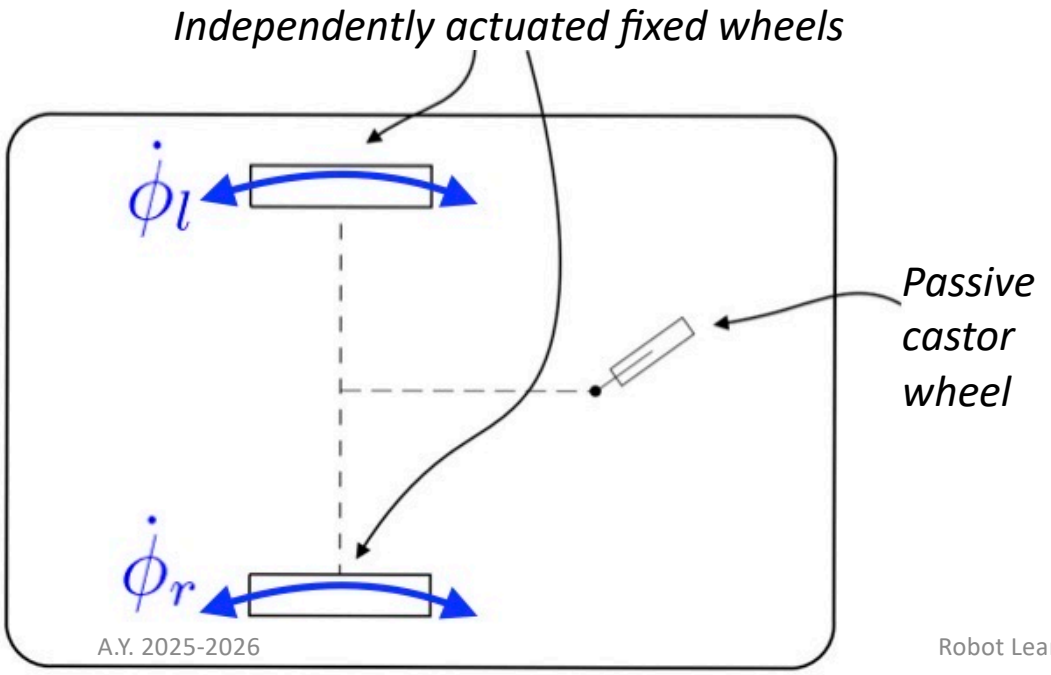
Differential drive

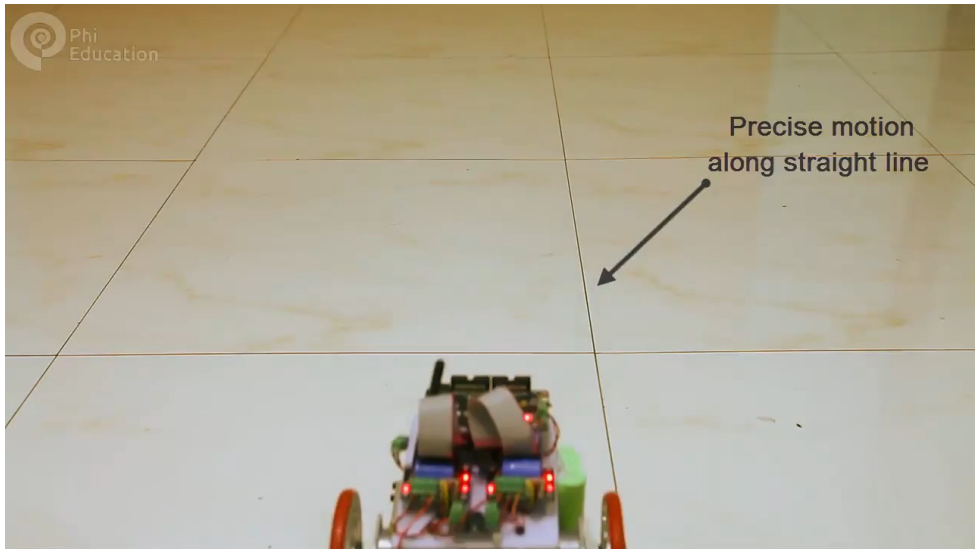
- A more realistic wheeled robot is the tri-cycle with differential drive, using two active fixed wheels and a front castor wheel.



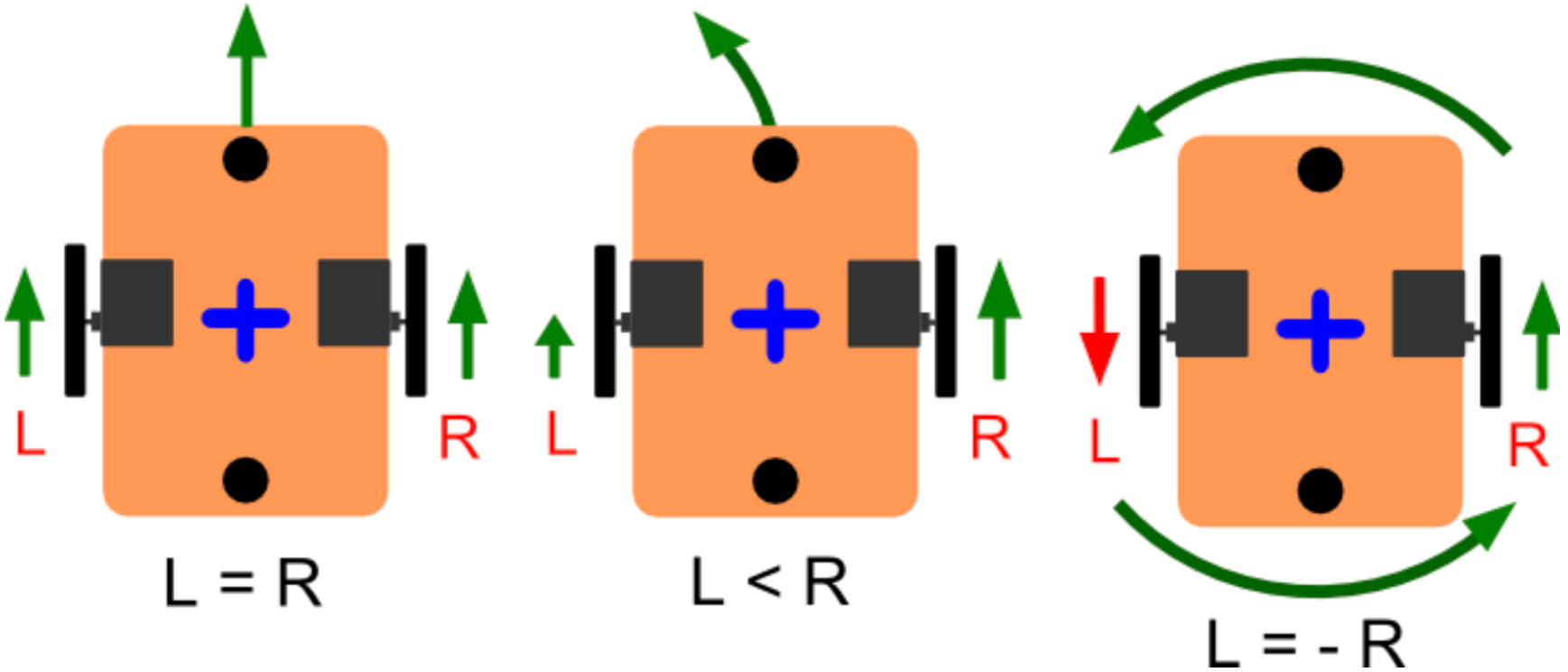
Differential drive

- A more realistic wheeled robot is the tri-cycle with differential drive, using two active fixed wheels and a front castor wheel.





Differential drive



Differential drive

World frame: $\{X_W, Y_W\}$

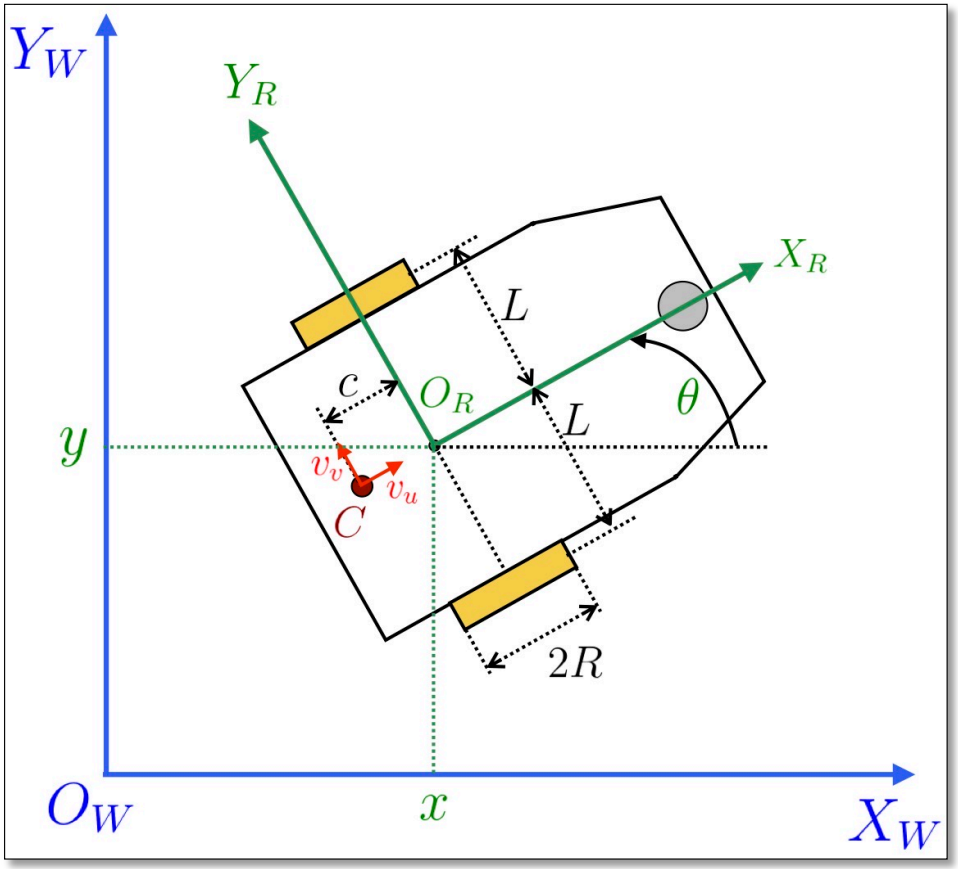
Body frame: $\{X_R, Y_R\}$

Assumption 1: the robot is **symmetric** along the longitudinal axis X_R

- Equidistant wheels (axle length is $2L$)
- Identical wheel radius ($R_L = R_R = R$)
- Center of mass is on X_R, at a distance c from O_R

Assumption 2: the robot is a **rigid body**

- The distance between any two points on the robot is fixed
- The distance c is fixed



Differential drive

The rotation of the robot is only around the vertical axis, i.e.,

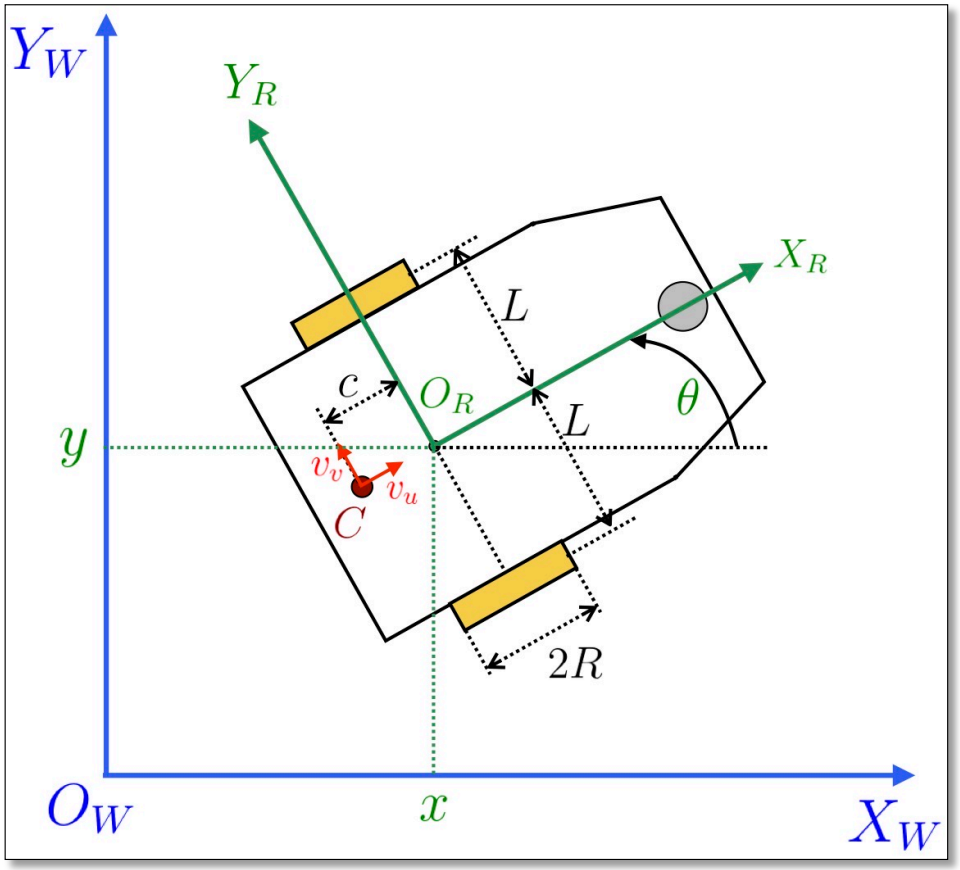
$$\boldsymbol{\omega} = [0,\ 0,\ \omega_z]$$

The velocity of the center of mass in body frame is

$${}^{R}\boldsymbol{v}_C = [v_u,\ v_v,\ 0]$$

$${}^{W}\boldsymbol{v}_A = \begin{bmatrix} \dot{x} \\ \dot{y} \\ 0 \end{bmatrix} = R_Z(\theta)\ {}^{R}\boldsymbol{v}_A$$

$$= \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_u \\ v_v - c\dot{\theta} \\ 0 \end{bmatrix}$$



Differential drive

Imposing the pure-rolling constraints, we have

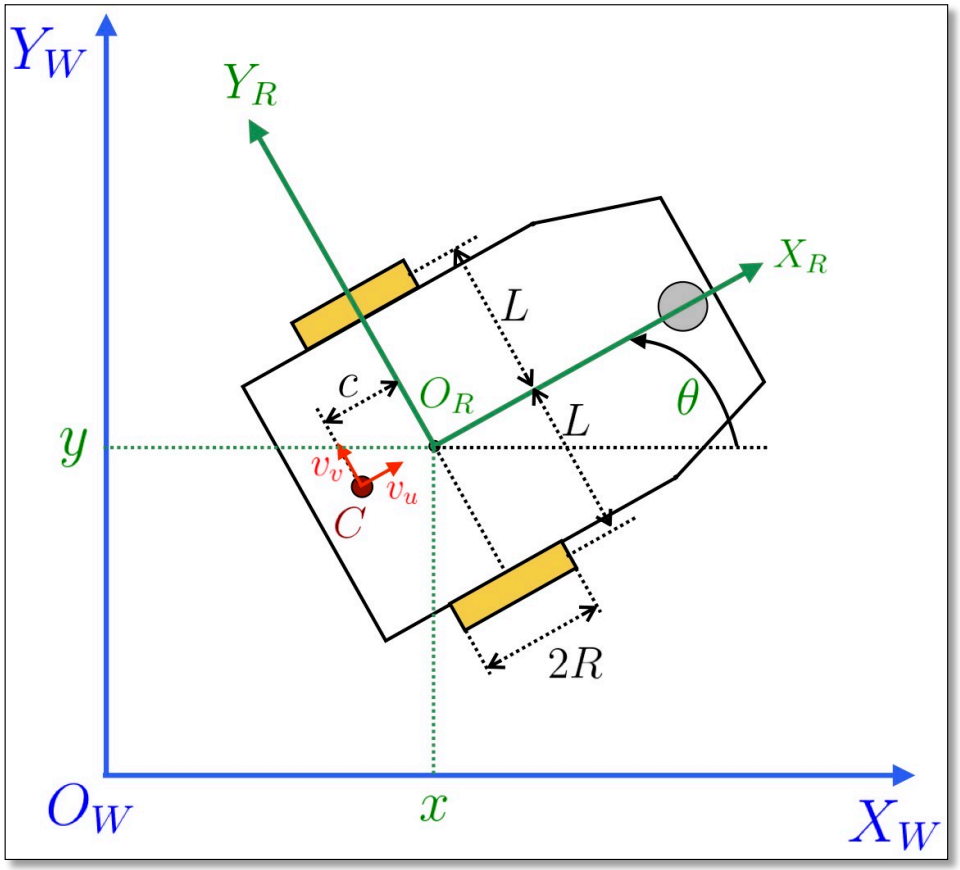
1. No skidding: The lateral velocity of the point O_R in the robot frame is null

$$ {}^R\boldsymbol{v}_A = \begin{bmatrix} v_u \\ \cancel{v_v - c\dot{\theta}} \\ 0 \end{bmatrix} = \begin{bmatrix} v_u \\ 0 \\ 0 \end{bmatrix} $$

2. No slipping: every full revolution a wheel travels a distance equal to its circumference

$$ v_l = R\,\dot{\phi}_l $$

$$ v_r = R\,\dot{\phi}_r $$



Differential drive

Putting it all together

1) $\boldsymbol{\omega} = [0,\ 0,\ \omega_z]$

2) $${}^{W}\boldsymbol{v}_A = \begin{bmatrix} \dot{x} \\ \dot{y} \\ 0 \end{bmatrix} = R_Z(\theta)\ {}^{R}\boldsymbol{v}_A$$

$$= \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_u \\ v_v - c\dot{\theta} \\ 0 \end{bmatrix}$$

3) $${}^{R}\boldsymbol{v}_A = \begin{bmatrix} v_u \\ v_v - c\dot{\theta} \\ 0 \end{bmatrix} = \begin{bmatrix} v_u \\ 0 \\ 0 \end{bmatrix}$$

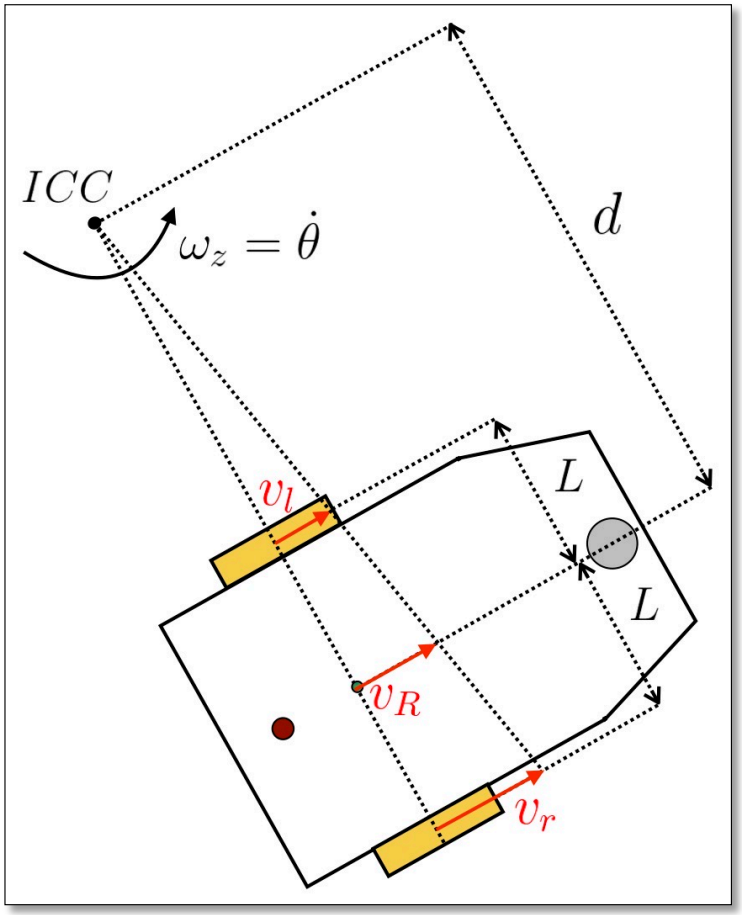
Equivalent to a unicycle model, with the wheel placed in O_R!

$$\dot{\boldsymbol{x}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & 0 \\ \sin(\theta) & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v_u \\ \omega_z \end{bmatrix}$$

Instantaneous center of rotation

No slipping condition -> all points have velocity orthogonal to the distance to the ICR

$$\begin{cases} v_l &= \omega_z \, (d - L) \\ v_R &= \omega_z \, d \\ v_r &= \omega_z \, (d + L) \end{cases} \quad \Rightarrow \quad d = L \frac{v_r + v_l}{v_r - v_l}$$



1. If $v_l = v_r$ => no turn (ICR is undefined)
2. If $v_r = - v_l$ => turn "on itself" ($ICR = O_R$)
3. If $v_r = 0$ (v_l = 0) => turn "on wheel" (d = -L or $d = L$)

Differential drive kinematics

No slipping

$$\begin{cases} v_l &= \omega_z \, (d - L) \\ v_R &= \omega_z \, d \\ v_r &= \omega_z \, (d + L) \end{cases}$$

No skidding

$$\begin{cases} v_l &= R\dot{\phi}_l \\ v_r &= R\dot{\phi}_r \end{cases} \Rightarrow \begin{cases} v_{Rx} &= \frac{v_r + v_l}{2} = \frac{R}{2}(\dot{\phi}_r + \dot{\phi}_l) \\ \omega_z &= \frac{v_r - v_l}{2L} = \frac{R}{2L}(\dot{\phi}_r - \dot{\phi}_l) \end{cases}$$

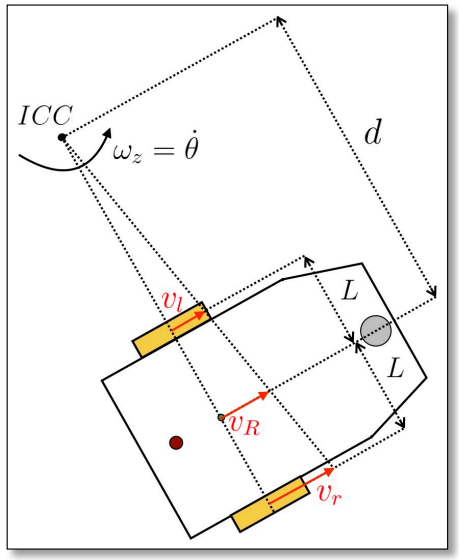
Differential Forward Kinematics

$$\dot{\boldsymbol{x}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & 0 \\ \sin(\theta) & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ \frac{1}{L} & -\frac{1}{L} \end{bmatrix} \underbrace{\begin{bmatrix} \dot{\phi}_l \\ \dot{\phi}_r \end{bmatrix}}_{\dot{\boldsymbol{q}}}$$

Differential drive kinematics

To invert the forward differential kinematics, the same procedure as for serial manipulators applies

We may also reason in the frame fixed to the robot, to find an easy relation

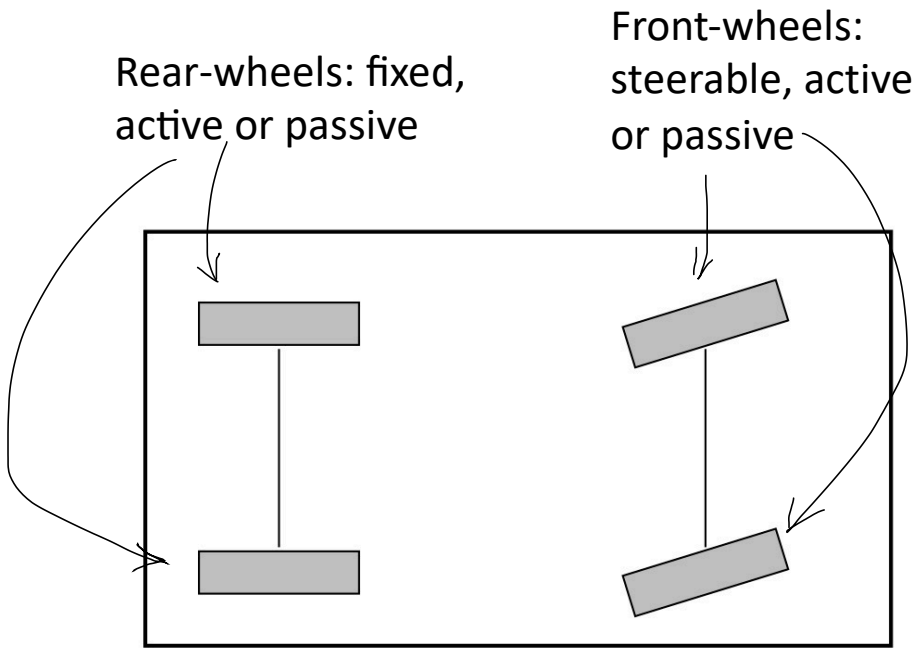


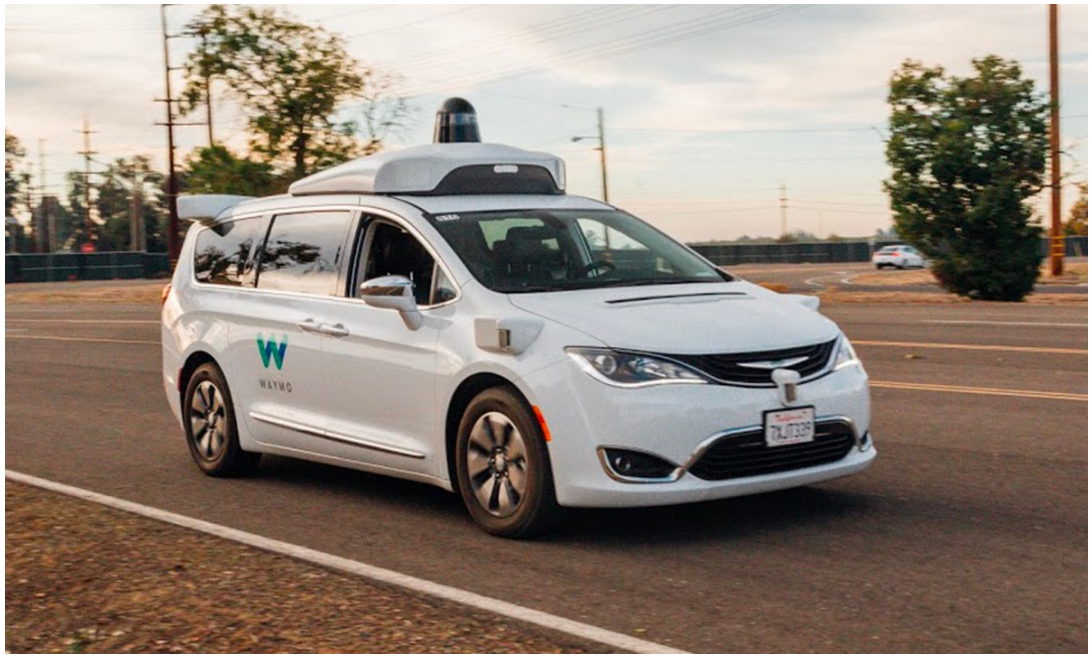
Inverse Differential Kinematics (in robot frame)

$$\begin{cases} v_{Rx} &= \frac{v_r + v_l}{2} = \frac{R}{2}(\dot{\phi}_r + \dot{\phi}_l) \\ \omega_z &= \frac{v_r - v_l}{2L} = \frac{R}{2L}(\dot{\phi}_r - \dot{\phi}_l) \end{cases} \Rightarrow \underbrace{\begin{bmatrix} \dot{\phi}_l \\ \dot{\phi}_r \end{bmatrix}}_{\dot{\boldsymbol{q}}} = \frac{1}{R}\begin{bmatrix} 1 & L \\ 1 & -L \end{bmatrix}\begin{bmatrix} v_{Rx} \\ \omega_z \end{bmatrix}$$

Car-like robots

- A most common kind of mobile robot is the car-like





Car-like robots

- A most common kind of mobile robot is the car-like

Car-like robots

World frame: $\{X_W, Y_W\}$

Body frame: $\{X_R, Y_R\}$

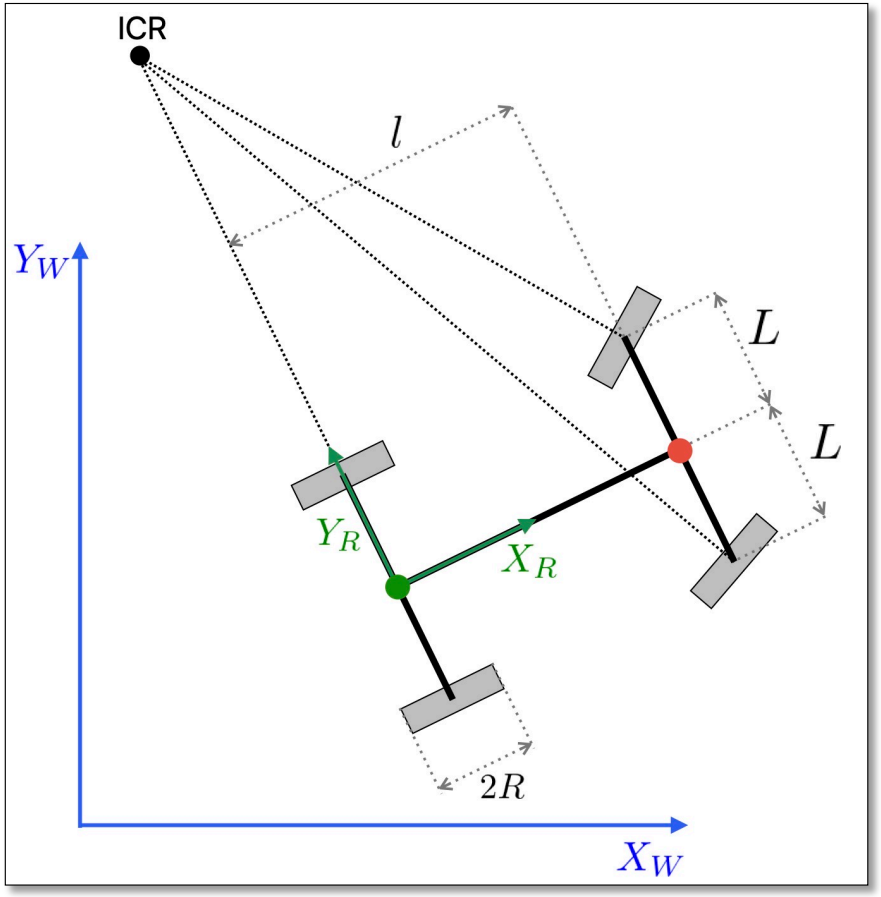
Assumption 1: the robot is **symmetric** along the longitudinal axis X_R

- Equidistant wheels (axle length is $2L$)
- Identical wheel radius ($R_L = R_R = R$)

Assumption 2: the robot is a **rigid body**

- The distance between any two points on the robot is fixed (in particular the distance between the rear and front axle is l)

Assumption 3: the rear wheels are **actuated**, the front wheels are **steerable**



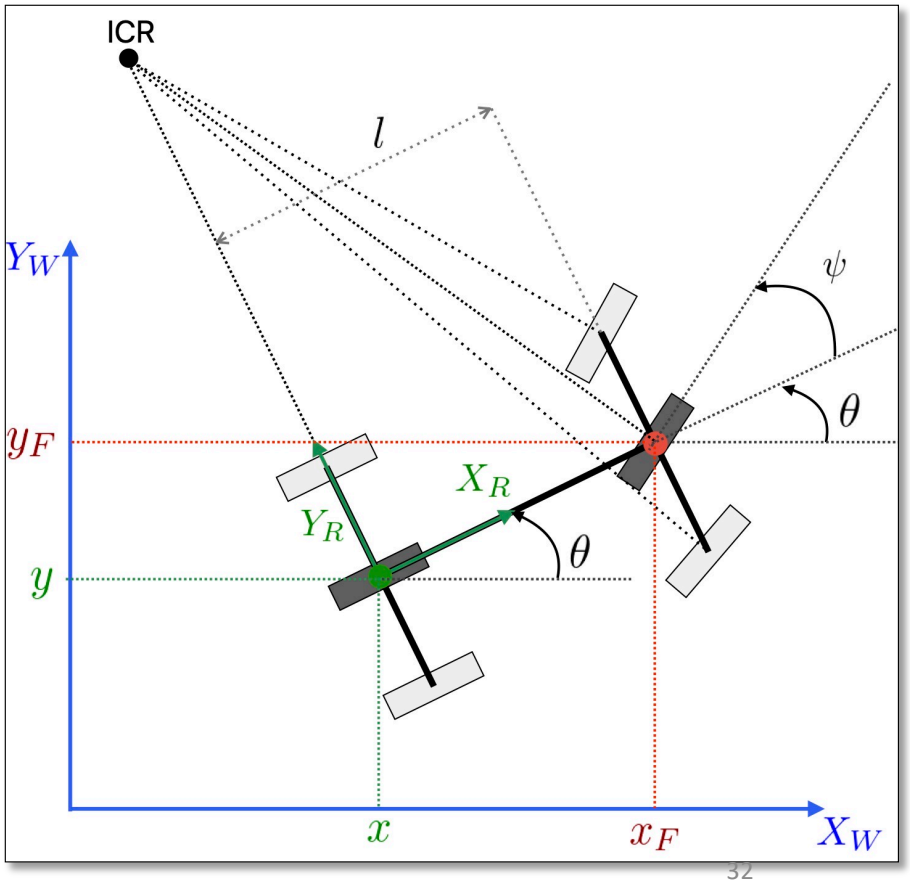
Car-like robots

We can collapse the rear and front wheels, to approximate the car-like with a **bicycle model**

- The center of the rear wheel of the bicycle model is at coordinates (x, y)
- The center of the front wheel of the bicycle model is at coordinates (x_F, y_F)

$$\begin{bmatrix} x_F \\ y_F \end{bmatrix} = \begin{bmatrix} x + l\cos(\theta) \\ y + l\sin(\theta) \end{bmatrix}$$

- The **steering angle** of the front wheel is ψ

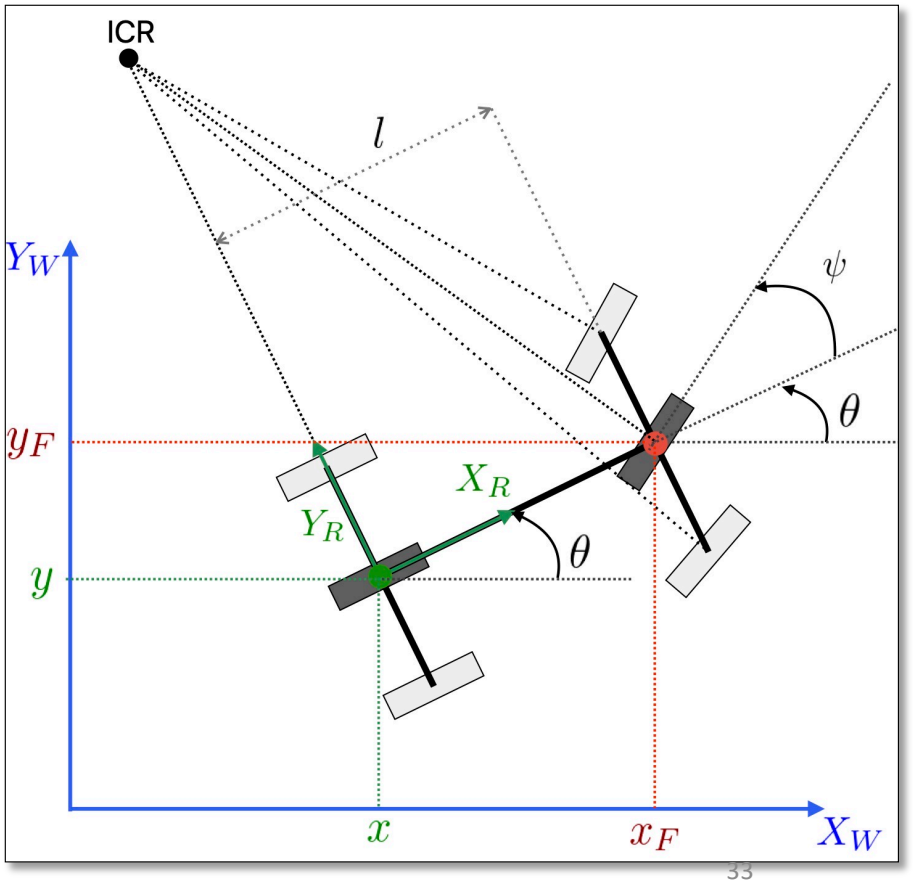


Car-like robots: non-holonomic constraints

Given the simplified bicycle mode, we must impose the no-slipping condition on both the rear and front wheels

- For the rear wheel, we have a constraint similar to the unicycle robot.

$$\begin{pmatrix} \sin(\theta) & -\cos(\theta) \end{pmatrix} \begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = \boldsymbol{n}^T \begin{pmatrix} \dot{x} \\ \dot{y} \end{pmatrix} = 0$$



Car-like robots: non-holonomic constraints

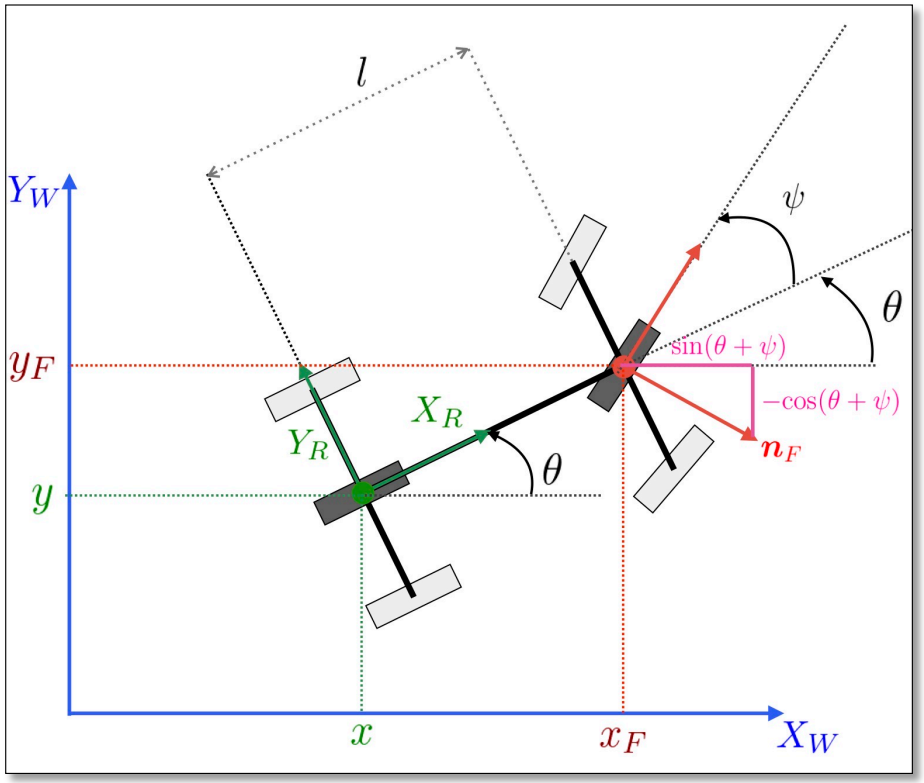
Given the simplified bicycle mode, we must impose the no-slipping condition on both the rear and front wheels

- For the front wheel, we can impose that the projection of the velocity of the front wheel on n_F is null.

$$\dot{x}_F \sin(\theta+\psi) - \dot{y}_F \cos(\theta+\psi) = 0$$

$$\Downarrow$$

$$\begin{bmatrix}\sin(\theta+\psi) & -\cos(\theta+\psi)\end{bmatrix} \begin{bmatrix}\dot{x}_F \\ \dot{y}_F\end{bmatrix} = \boldsymbol{n}_F^T \begin{bmatrix}\dot{x}_F \\ \dot{y}_F\end{bmatrix} = 0$$



Car-like robots: non-holonomic constraints

1. ***Rear wheel no-slipping constraint***

$$\dot{x}\,\sin(\theta) - \dot{y}\,\cos(\theta) = 0$$

2. Front wheel no-slipping constraint

$$\dot{x}_F\,\sin(\theta+\psi) - \dot{y}_F\,\cos(\theta+\psi) = 0$$

3. Front wheel position

$$\begin{bmatrix} x_F \\ y_F \end{bmatrix} = \begin{bmatrix} x + l\,\cos(\theta) \\ y + l\,\sin(\theta) \end{bmatrix}$$

4. Front wheel velocity

$$\Rightarrow \quad \begin{bmatrix} \dot{x}_F \\ \dot{y}_F \end{bmatrix} = \begin{bmatrix} \dot{x} - l\,\dot{\theta}\,\sin(\theta) \\ \dot{y} + l\,\dot{\theta}\,\cos(\theta) \end{bmatrix}$$

5. Front wheel no-slipping constraint highlighting $\dot{\theta}$

By plugging the front wheel velocity in the front wheel constraint, we can rewrite it as:

$$\begin{aligned} 0 &= \dot{x}_F\,\sin(\theta+\psi) - \dot{y}_F\,\cos(\theta+\psi) \\ &= \dot{x}\,\sin(\theta+\psi) - \dot{y}\,\cos(\theta+\psi) \\ &\underbrace{-l\,\dot{\theta}\,\sin(\theta)\,\sin(\theta+\psi) - l\,\dot{\theta}\,\cos(\theta)\,\cos(\theta+\psi)}_{-l\,\dot{\theta}\,\cos(\theta-\theta+\psi) = -l\,\dot{\theta}\,\cos(\psi)} \end{aligned}$$

Car-like robots: non-holonomic constraints

Combining both constraints, we can write them in the Pfaffian form

$$\begin{matrix}\textit{Rear wheel}\\ \textit{Front wheel}\end{matrix}\begin{bmatrix}\sin(\theta) & -\cos(\theta) & 0 & 0\\ \sin(\theta+\psi) & -\cos(\theta+\psi) & -l\,\dot{\theta}\cos(\psi) & 0\end{bmatrix}\begin{bmatrix}\dot{x}\\ \dot{y}\\ \dot{\theta}\\ \dot{\psi}\end{bmatrix} = A(\boldsymbol{q})\,\dot{\boldsymbol{q}} = \mathbf{0}$$

generalized coordinates

The admissible generalized velocities must belong to the null-space of $A(\boldsymbol{q})$.

- One basis of the null space is trivially $v_1 = \begin{bmatrix}0\\0\\0\\1\end{bmatrix}$
- To satisfy the rear wheel constraint, the second basis elem. must be in the form $v_2 = \begin{bmatrix}\cos(\theta)\\ \sin(\theta)\\ t\\ 0\end{bmatrix}$ where t is an incognita

Car-like robots: non-holonomic constraints

To find t, we impose that v_2 is orthogonal to the second row of $A(\boldsymbol{q})$

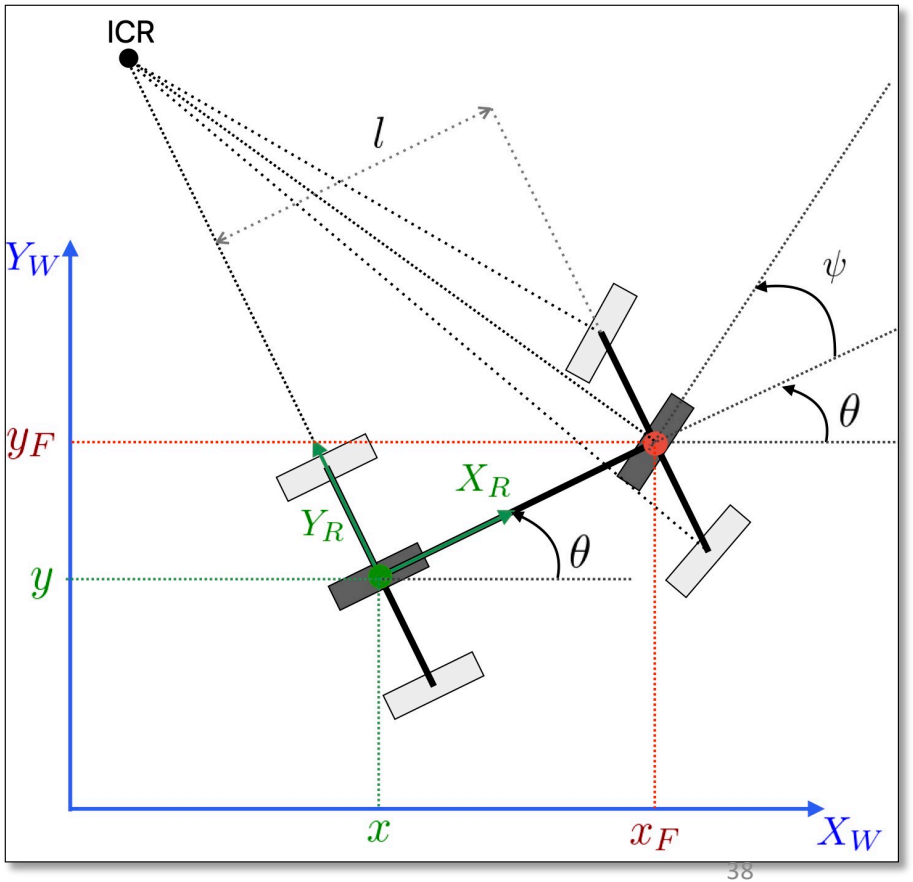
$$\begin{aligned}\sin(\theta+\psi)\cos(\theta) - \sin(\theta+\psi)\cos(\theta) - l\,\dot{\theta}\cos(\psi)t = \\ = \sin(\theta+\psi-\theta) - l\,\dot{\theta}\cos(\psi)t = \\ = \sin(\psi) - l\,\dot{\theta}\cos(\psi)t = 0\end{aligned}$$

$$t\,\dot{\theta} = \frac{\tan(\psi)}{l} \quad \Rightarrow \quad v_2 = \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \\ \frac{\tan(\theta)}{l} \\ 0 \end{bmatrix}$$

Car-like robots: kinematics

The kinematics model of the car-like robot is:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & 0 \\ \sin(\theta) & 0 \\ \frac{\tan(\psi)}{l} & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$



- u_1 is the **translational velocity input**. Under the assumption of equal wheels, we have that

$$u_1 = R\dot{\phi}_l = R\dot{\phi}_r$$

- u_2 is the **steering input**.

Car-like robots: kinematics of C.o.M.

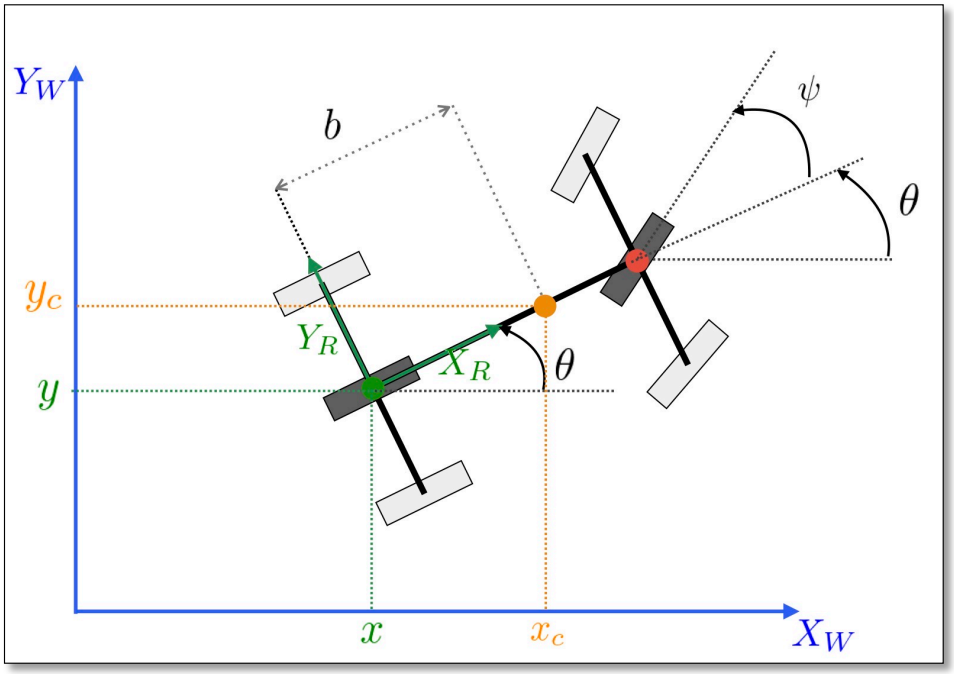
What about the motion of the **center of mass**?

From our assumptions, the center of mass is on X_R. Let's assume is at a distance b from the rear axle.

$$\begin{bmatrix} x_c \\ y_c \end{bmatrix} = \begin{bmatrix} x + b\cos(\theta) \\ y + b\sin(\theta) \end{bmatrix} \Rightarrow \begin{bmatrix} \dot{x}_c \\ \dot{y}_c \end{bmatrix} = \begin{bmatrix} \dot{x} - b\dot{\theta}\sin(\theta) \\ \dot{y} + b\dot{\theta}\cos(\theta) \end{bmatrix}$$

Using the expressions that we have derived for $\dot{x},\ \dot{y},\ \dot{\theta}$

$$\Rightarrow \begin{bmatrix} \dot{x}_c \\ \dot{y}_c \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -b\dfrac{\tan(\psi)}{l}\sin(\theta) \\ \sin(\theta) & b\dfrac{\tan(\psi)}{l}\cos(\theta) \\ \dfrac{\tan(\psi)}{l} & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$



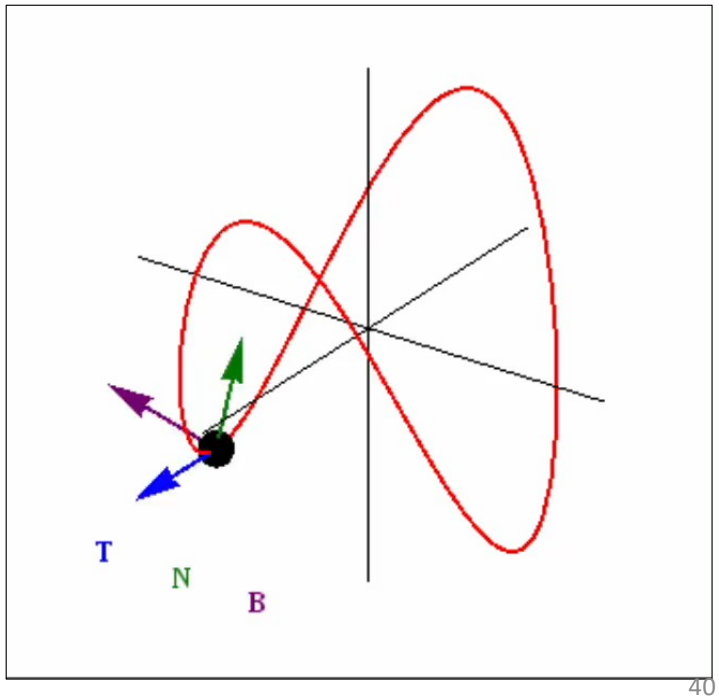
Frénet frame kinematics

We can also derive the car-like kinematics using other generalized coordinates. For example, for path tracking it is convenient to refer the coordinates to a Frénet frame

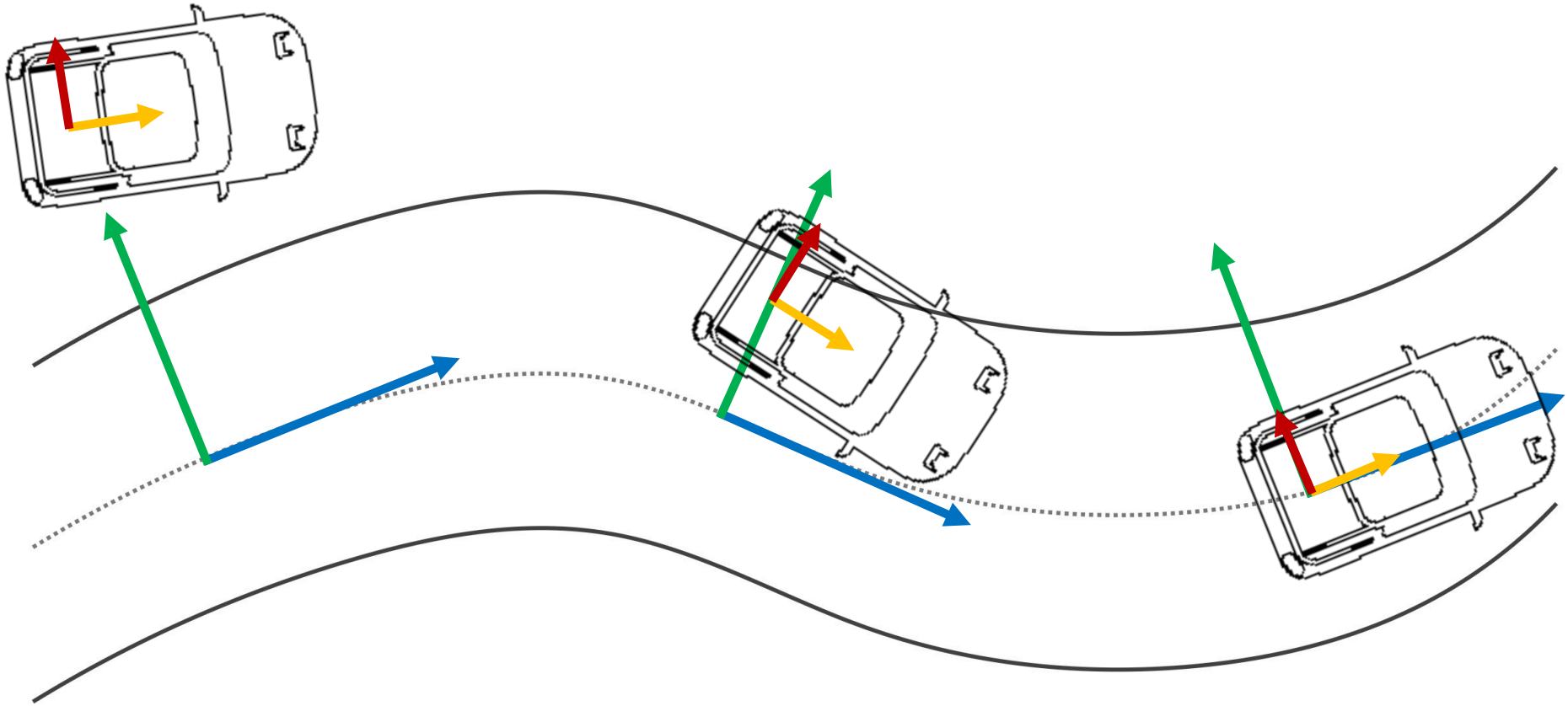
- $r(t)$ is a curve in Euclidean space
- The **curvilinear abscissa** is $s(t) = \int_0^t \|r'(\sigma)\| d\sigma$

The ***Frénet frame*** at a certain point of the curve identified by s is given by three vectors

- T is the **tangent vector** to the curve in s: $T = \frac{dr}{ds}$
- N is the **normal vector** to the curve in s: $N = \frac{\frac{dT}{ds}}{\|\frac{dT}{ds}\|}$
- B is the **binormal vector** to the curve in s: $B = T \times N$



Frénet frame kinematics



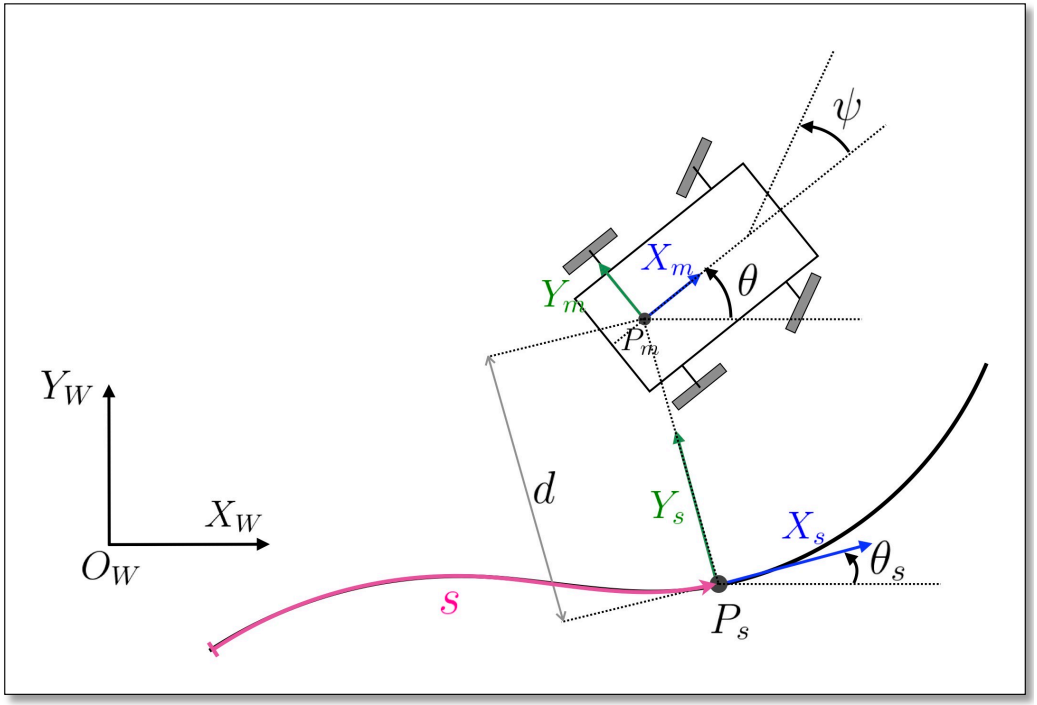
Frénet frame kinematics

Consider three frames of reference:

- A fixed world frame: $F_W = \{O_W, X_W, Y_W\}$
- A frame fixed on the robot: $F_m = \{P_m, X_m, Y_m\}$
- A Frénet frame on the path: $F_s = \{P_s, X_s, Y_s\}$

Definitions:

- **Distance** of the point P_m from P_s, along Y_s at s: d
- **Curvature** of the path at s: $\kappa(s) = \dfrac{\partial \theta_s}{\partial s}$
- **Orientation** of the robot w.r.t. F_s: $\theta_e = \theta - \theta_s$



Kinematics in the Frénet Frame

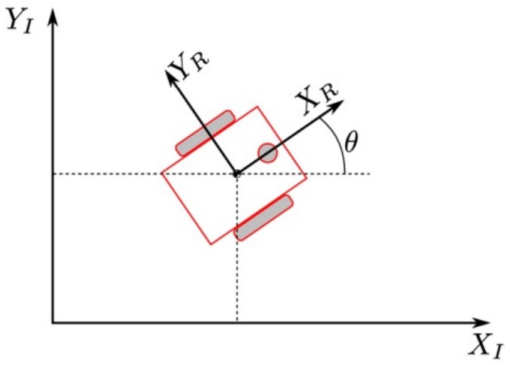
$$\begin{cases} \dot{s} &= \dfrac{\cos(\theta_e)}{1 - d\,\kappa(s)} u_1 \\ \dot{d} &= \sin(\theta_e) u_1 \\ \dot{\theta}_e &= -\dot{s}\,\kappa(s) + \dfrac{\tan(\psi)}{l} u_1 \\ \dot{\psi} &= u_2 \end{cases}$$

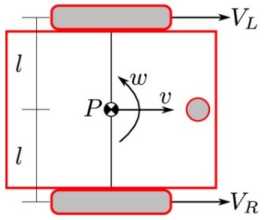
Integrating differential kinematics

Dead reckoning: Since for wheeled mobile robots we don't have a direct kinematic relation that maps the configuration variables $\boldsymbol{q}$ to the task space $\boldsymbol{x}$, we can determine the pose by **integrating** the **differential kinematics**, via the estimate of the wheels' speeds $(\dot{\phi}_R,\ \dot{\phi}_L)$

Example

For a two-wheeled differential robot:



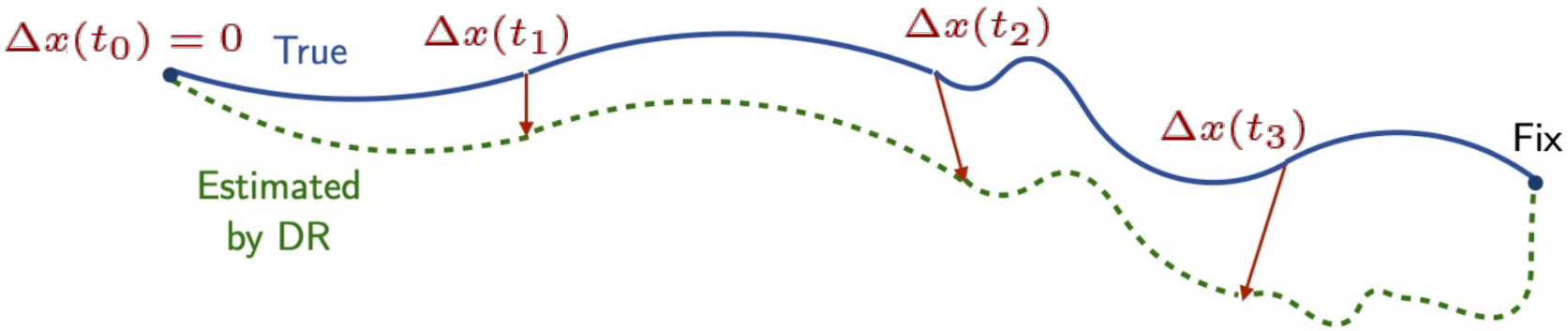


$$x(t) = x(t_o) + \frac{R}{2}\int_{t_0}^{t_f} (\dot{\phi}_R(t) + \dot{\phi}_L(t))\cos(\theta(t))\,dt$$

$$y(t) = y(t_o) + \frac{R}{2}\int_{t_0}^{t_f} (\dot{\phi}_R(t) + \dot{\phi}_L(t))\sin(\theta(t))\,dt$$

$$\theta(t) = \theta(t_0) + \frac{R}{2l}\int_{t_0}^{t_f} (\dot{\phi}_R(t) + \dot{\phi}_L(t))\,dt$$

Integrating differential kinematics



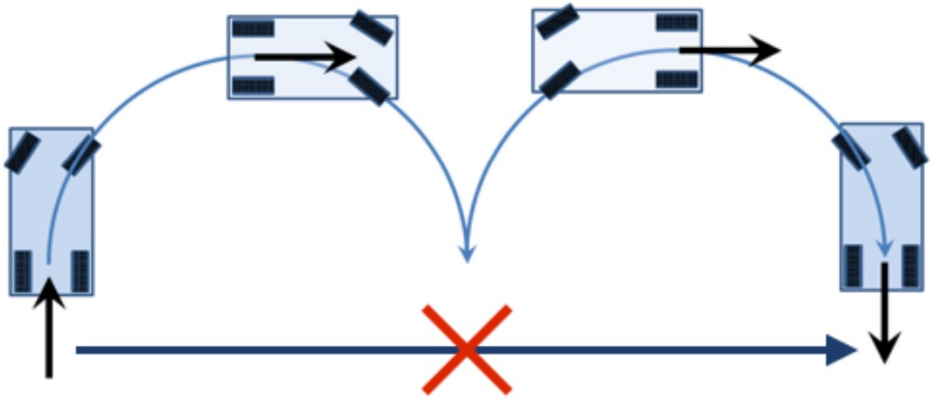
- The uncertainty of dead reckoning (odometry integraton increases over time). Causes are:
  - Noise in the measurements
  - Numerical integration errors
  - Wheel slippage
  - Inaccurate calibration

- A new **pose fix** is needed intermittently to restart the integration and avoid the estimate drift. For example, using a Kalman Filter to fuse the odometry velocities read via relative encoders at the wheels, with a slower GPS measurement

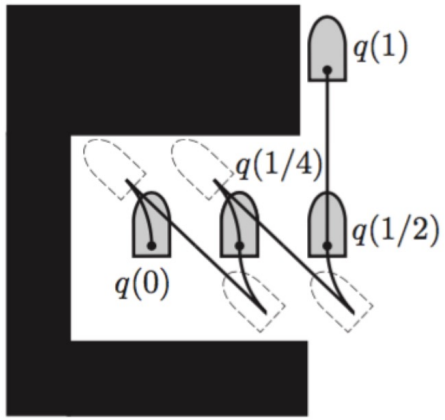
Kinodynamic planning

The presence of non-holonomic constraints limits the **maneuverability** of wheeled mobile robots. To plan the motion to a desired final pose, we must take into account these kinematic constraints.

If a planning problem involves constraints on at least velocity and acceleration, the problem is often referred to as **kinodynamic planning**.



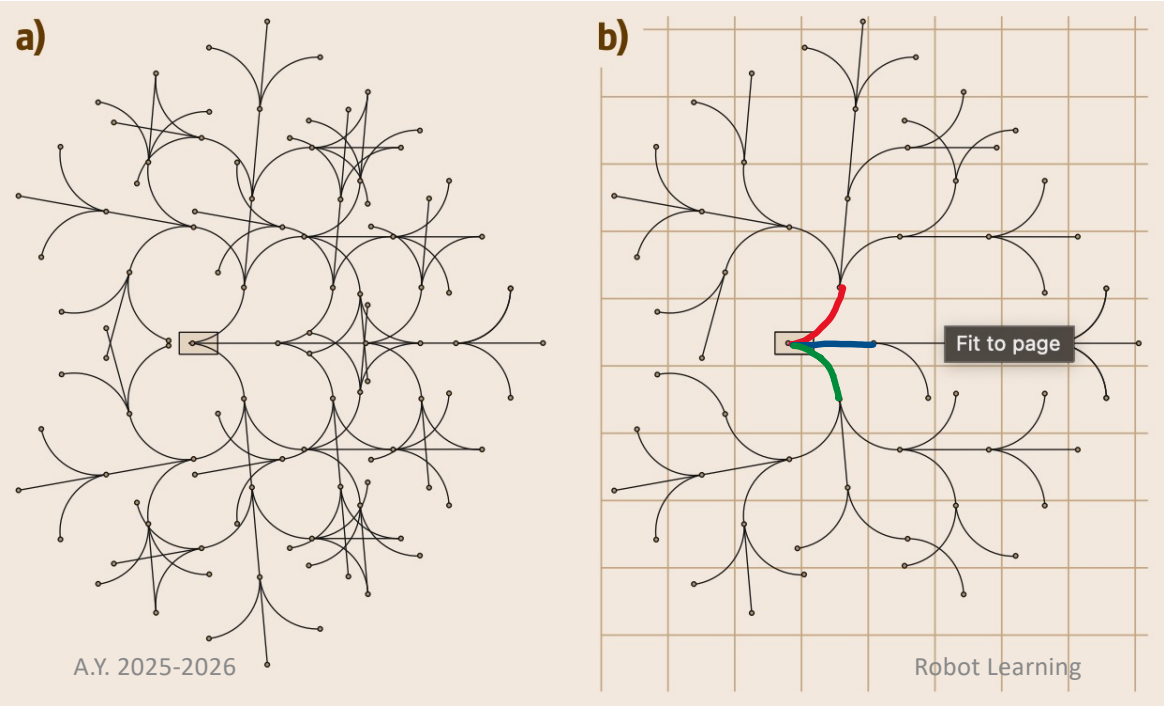
Two-moves car parking:
no side-way motion



Constraints on the
maximum turning angle

Kinodynamic planning

Due to the great difficulty of planning under differential constraints, many successful planning algorithms that address kinodynamic problems directly in the task space X are **sampling based** (e.g., Probabilistic RoadMap (**PRM**), Rapidly exploring Random Trees (**RRT**))



With a sampling-based method, we can grow a **search tree** in the task space using the **motion primitives** made available by the kinematic model of the wheeled robot. For example:

- Go straight
- Forward left
- Forward right

Kinodynamic planning

Instead of using a fixed discretized number of motion primitives, we can also use **probabilistic motion primitives** (**ProMP**)

- A motion is encoded as a **linear combination** of some **weighted basis functions**
- the weight vector is assumed to have a normal distribution

Basis functions Φ

Weight vector $\tilde{\mu} \sim \mathcal{N}\{\mu, \Sigma\}$

$$\tilde{\xi} = \Phi \tilde{\mu}$$

Sampled trajectory

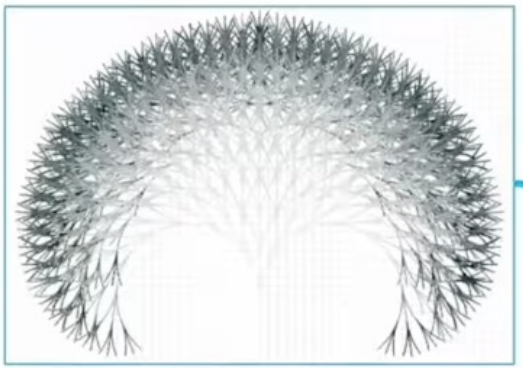
The **motion primitive distribution** can be generated by using the **forward kinematics of the wheeled robot**:

1. Discretize the input space (i.e., the control commands)
2. Record the output (i.e, the trajectory) of the forward kinematics,
3. Calculate the mean μ and covariance Σ of the set of trajectories

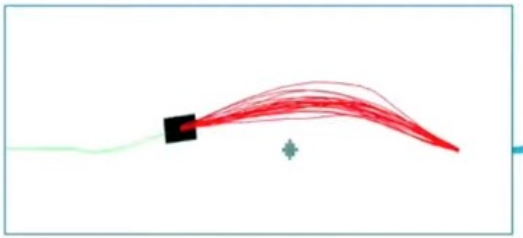
"***Probabilistic Motion Primitives based Trajectory Planning***", RSS 2021, Spotlight https://www.roboticsproceedings.org/rss17/p058.pdf

Kinodynamic planning

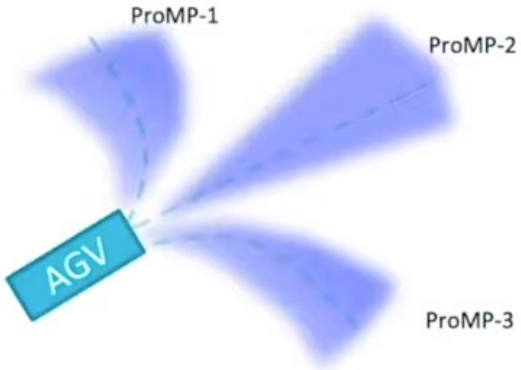
- **Lattice planner**: uses a set of fixed sampled motion primitives. This introduces a discretization problem
- **Trajectory optimization**: a trajectory can be represented as a parametric function and we may optimize its parameters, e.g., to avoid obstacles. But this may require high sampling rate in cluttered environments and it can become infeasible in real-time.



Lattice planner Pivtoraiko et.al 2007



STOMP, Kalakrishnan et.al, 2011



Probabilistic Motion Primitives (ProMP)

"***Probabilistic Motion Primitives based Trajectory Planning***", RSS 2021,
Source: https://www.roboticsproceedings.org/rss17/p058.pdf , Video: https://youtu.be/-C_oT4bpQUg?si=NFF5c1QOIyfGN9Lp

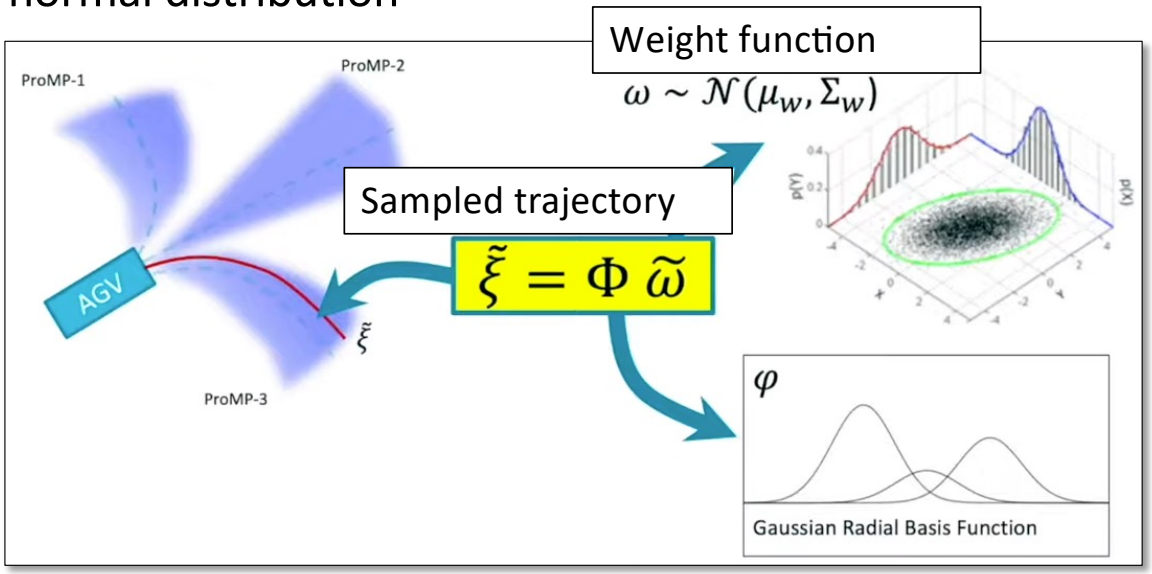
Kinodynamic planning

Instead of using a fixed discretized number of motion primitives, we can also use **probabilistic motion primitives** (**ProMP**)

- A motion is encoded as a **linear combination** of some **weighted basis functions**
- the weight vector is assumed to have a normal distribution

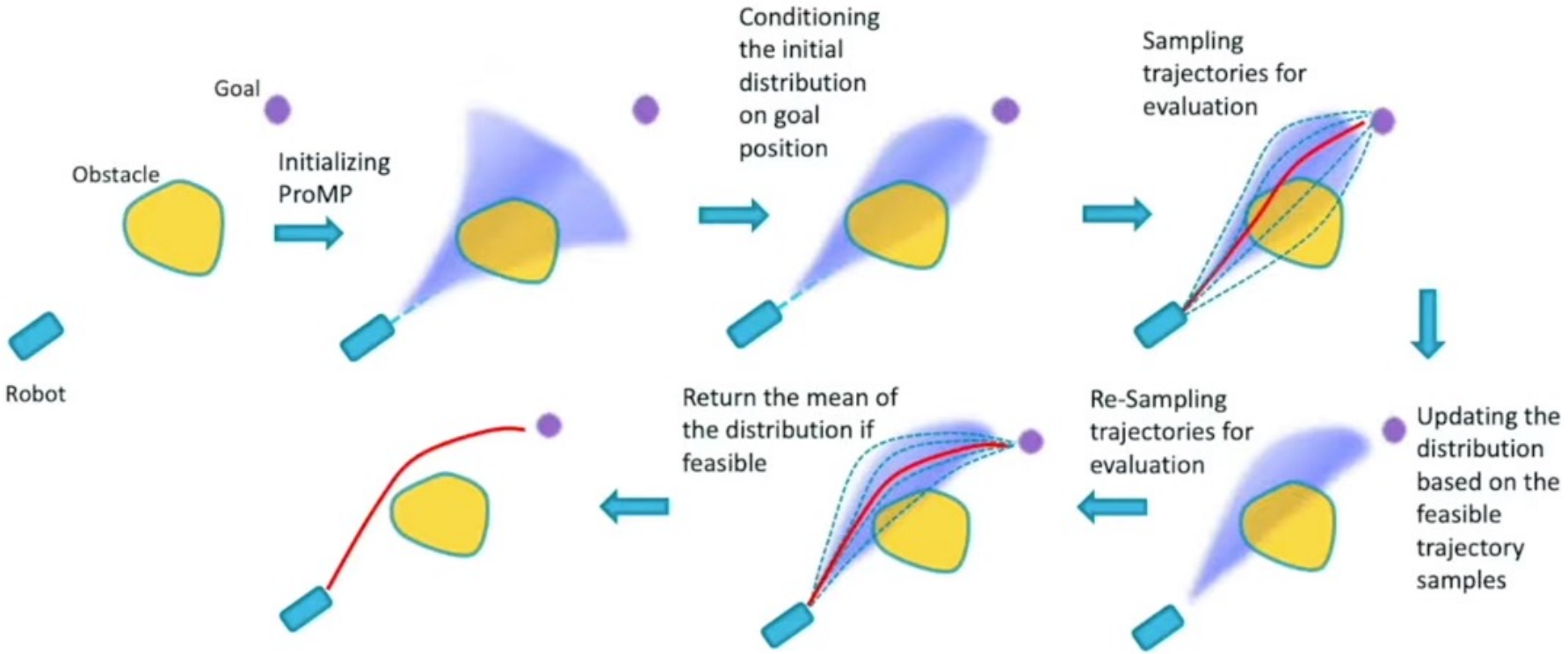
The **motion primitive distribution** can be generated by using the **forward kinematics of the wheeled robot**:

1. Discretize the input space (i.e., the control commands)
2. Record the output (i.e, the trajectory) of the forward kinematics,
3. Calculate the mean μ and covariance Σ of the set of trajectories



"***Probabilistic Motion Primitives based Trajectory Planning***", RSS 2021,
Source https://www.roboticsproceedings.org/rss17/p058.pdf

Kinodynamic planning



"***Probabilistic Motion Primitives based Trajectory Planning***", RSS 2021,
Source https://www.roboticsproceedings.org/rss17/p058.pdf

Dynamics

Dynamics

To compute the dynamics of a wheeled robots, we can use **Lagrange method** as done for serial manipulators, by solving

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_i}\right) - \frac{\partial L}{\partial q_i} = f_i + A(q)^T \lambda$$

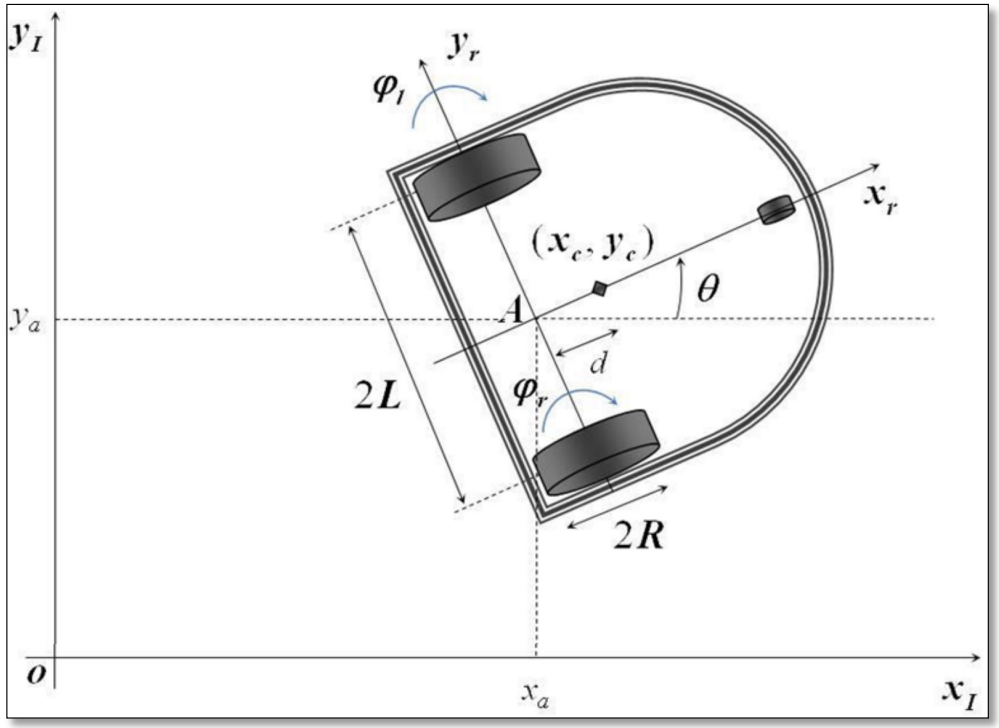
No-slipping constraint of the wheels

Lagrange multipliers

Example: differential drive robot

Assumptions:

- The mass is located along x_r, at a distance d from the axle point A.
- The wheels have no mass
- The wheels have no-slip and no-skid



Preliminaries:

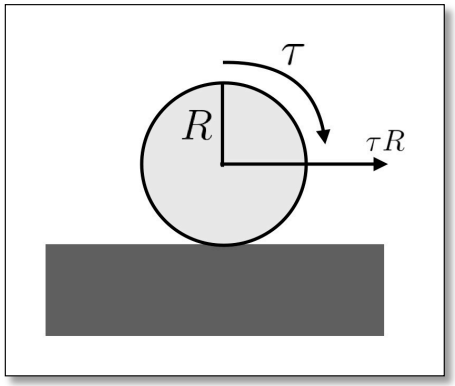
- The generalized coordinates are $\boldsymbol{q} = [x_a, y_a, \theta]$
- The coordinates of the center of mass are $\begin{cases} x_c = x_a + d\cos(\theta) \\ y_c = y_a + d\sin(\theta) \end{cases}$
- The velocity of the center of mass is $v_c = \sqrt{(\dot{x}_c^2 + \dot{y}_c^2)}$

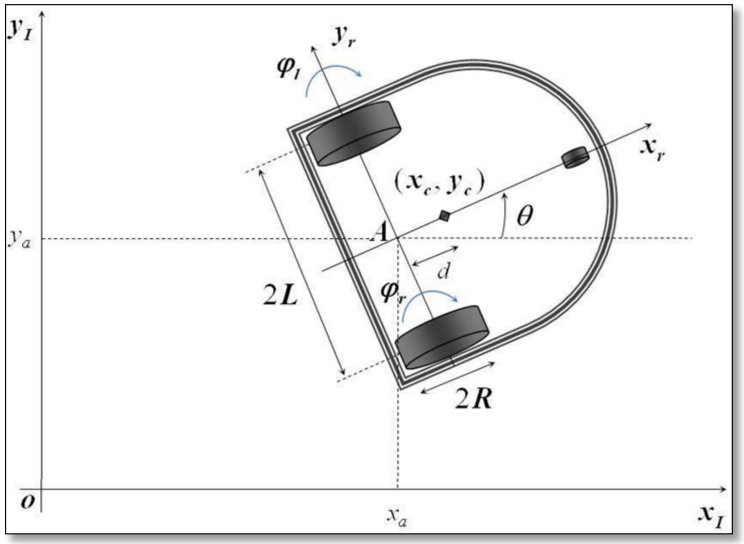
Example: differential drive robot

Preliminaries:

- The generalized coordinates forces are:

$$f_1 = \frac{1}{R}(\tau_r + \tau_l)\cos(\theta)$$
$$f_2 = \frac{1}{R}(\tau_r + \tau_l)\sin(\theta)$$
$$f_r = \frac{L}{R}(\tau_r - \tau_l)$$





- The non-holonomic constraint matrix is:

$$A(q) = \begin{bmatrix} \sin(\theta) & -\cos(\theta) & 0 \end{bmatrix}$$

- The Lagrangian is: $L = T - \underbrace{U}_{=0} = T$

Example: differential drive robot

Kinetic energy: $T = \frac{1}{2} m_c \, v_c^2 + \frac{1}{2} I_c \dot{\theta}^2 = L$

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_i}\right) - \frac{\partial L}{\partial q_i} = f_i + A(q)^T \lambda$$

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{x}_a}\right) = m_c \ddot{x}_a - m_c d \sin(\theta)\ddot{\theta} - m_c d \cos(\theta)\dot{\theta}^2 \qquad \frac{\partial L}{\partial x_a} = 0$$

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{y}_a}\right) = m_c \ddot{y}_a + m_c d \cos(\theta)\ddot{\theta} - m_c d \sin(\theta)\dot{\theta}^2 \qquad \frac{\partial L}{\partial y_a} = 0$$

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}_a}\right) = (m_c d^2 + I_c)\ddot{\theta} - m_c d \sin(\theta)\ddot{x}_a + m_c d \cos(\theta)\ddot{y}_a - m_c d \cos(\theta)\dot{x}_a\dot{\theta} - m_c d \sin(\theta)\dot{y}_a\dot{\theta}$$

$$\frac{\partial L}{\partial \theta} = -m_c d \cos(\theta)\dot{x}_a\dot{\theta} - m_c d \sin(\theta)\dot{y}_a\dot{\theta}$$

Example: differential drive robot

Putting everything together we get

$$B(\boldsymbol{q})\ddot{\boldsymbol{q}} + C(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} = \boldsymbol{f} + A(\boldsymbol{q})^T \lambda$$

With:

$$B(\boldsymbol{q}) = \begin{bmatrix} m_c & 0 & -m_c d \sin(\theta) \\ 0 & m_c & m_c d \cos(\theta) \\ -m_c d \sin(\theta) & m_c d \cos(\theta) & (m_c d^2 + I_c) \end{bmatrix}$$

$$C(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \begin{bmatrix} 0 & 0 & -m_c d \cos(\theta)\dot{\theta} \\ 0 & 0 & -m_c d \sin(\theta)\dot{\theta} \\ 0 & 0 & 0 \end{bmatrix}$$

$$\boldsymbol{f}(\theta) = \begin{bmatrix} \frac{1}{R}(\tau_r + \tau_l)\cos(\theta) \\ \frac{1}{R}(\tau_r + \tau_l)\sin(\theta) \\ \frac{L}{R}(\tau_r - \tau_l) \end{bmatrix}$$

$$A(\boldsymbol{q})^T = \begin{bmatrix} \sin(\theta) \\ -\cos(\theta) \\ 0 \end{bmatrix}$$

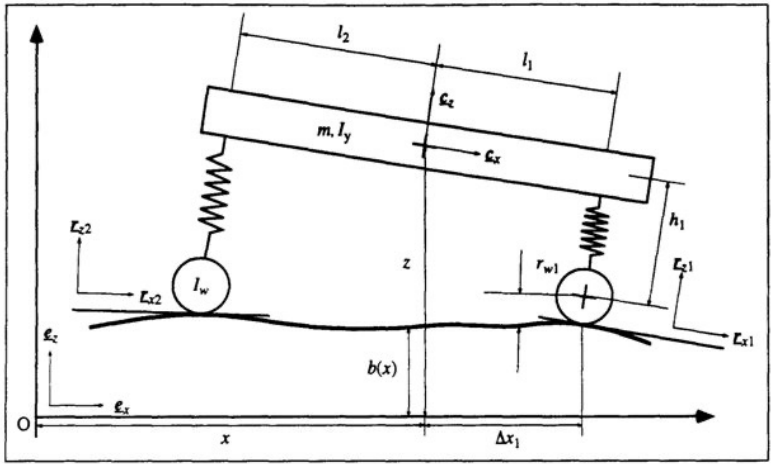
Simple vs. complex models

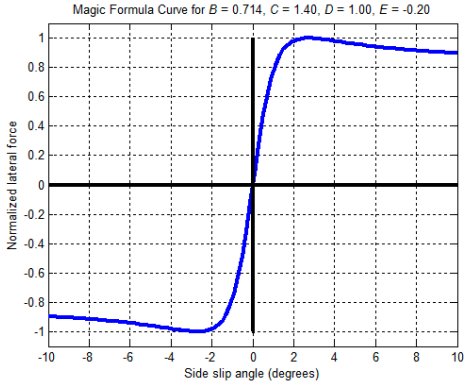
Simple vs. complex models

Simple vs. complex models

For an autonomous driving car or for racing applications we may need to consider a more complex model

- Wheels with non-negligible mass
- Actuators
- Tires interaction with possibly uneven floor (**Pacejka Magic Formula** or other models)
- Suspensions and other moving parts
- Aerodynamic effects
- …



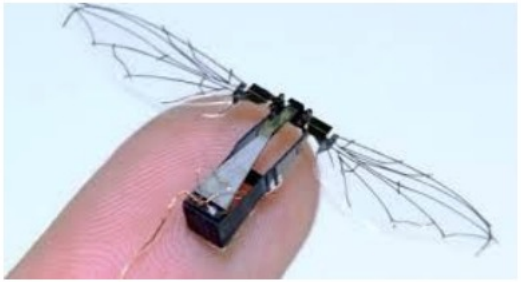


Open source simulators: CARLA

CARLA simulator https://carla.org/

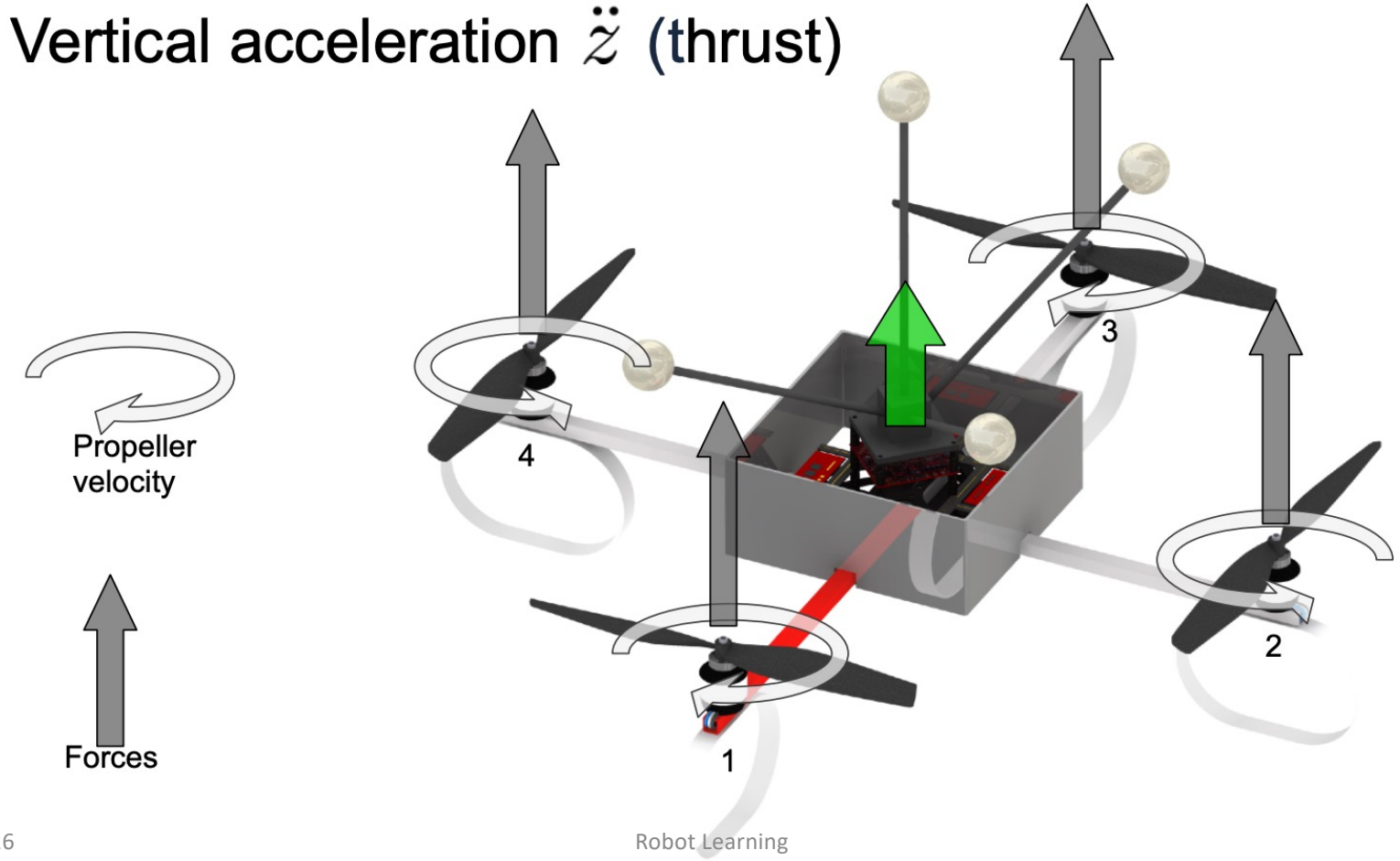
Aerial robots

Types of aerial robots

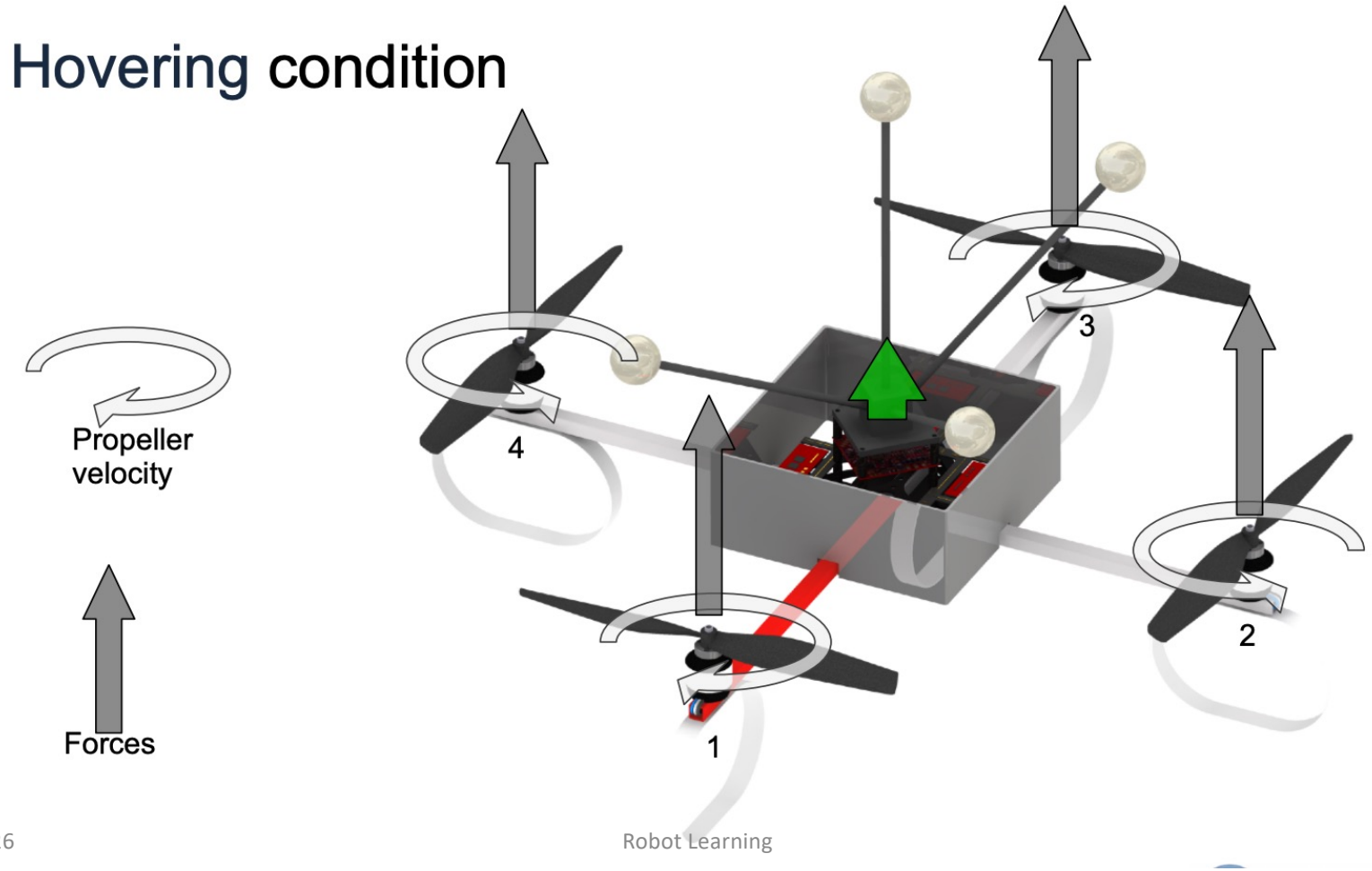


Quadrotor

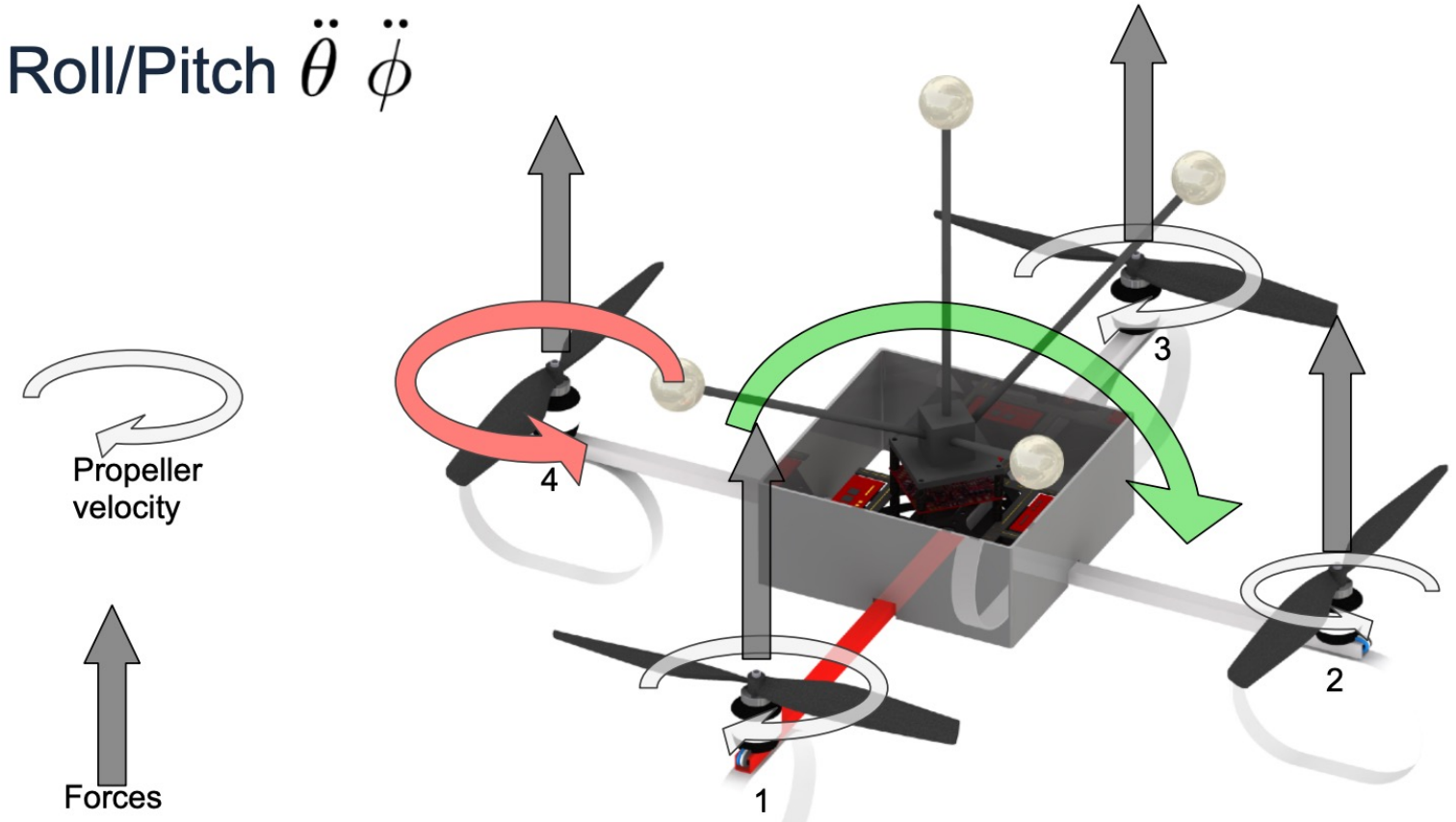
Quadrotor: working principle



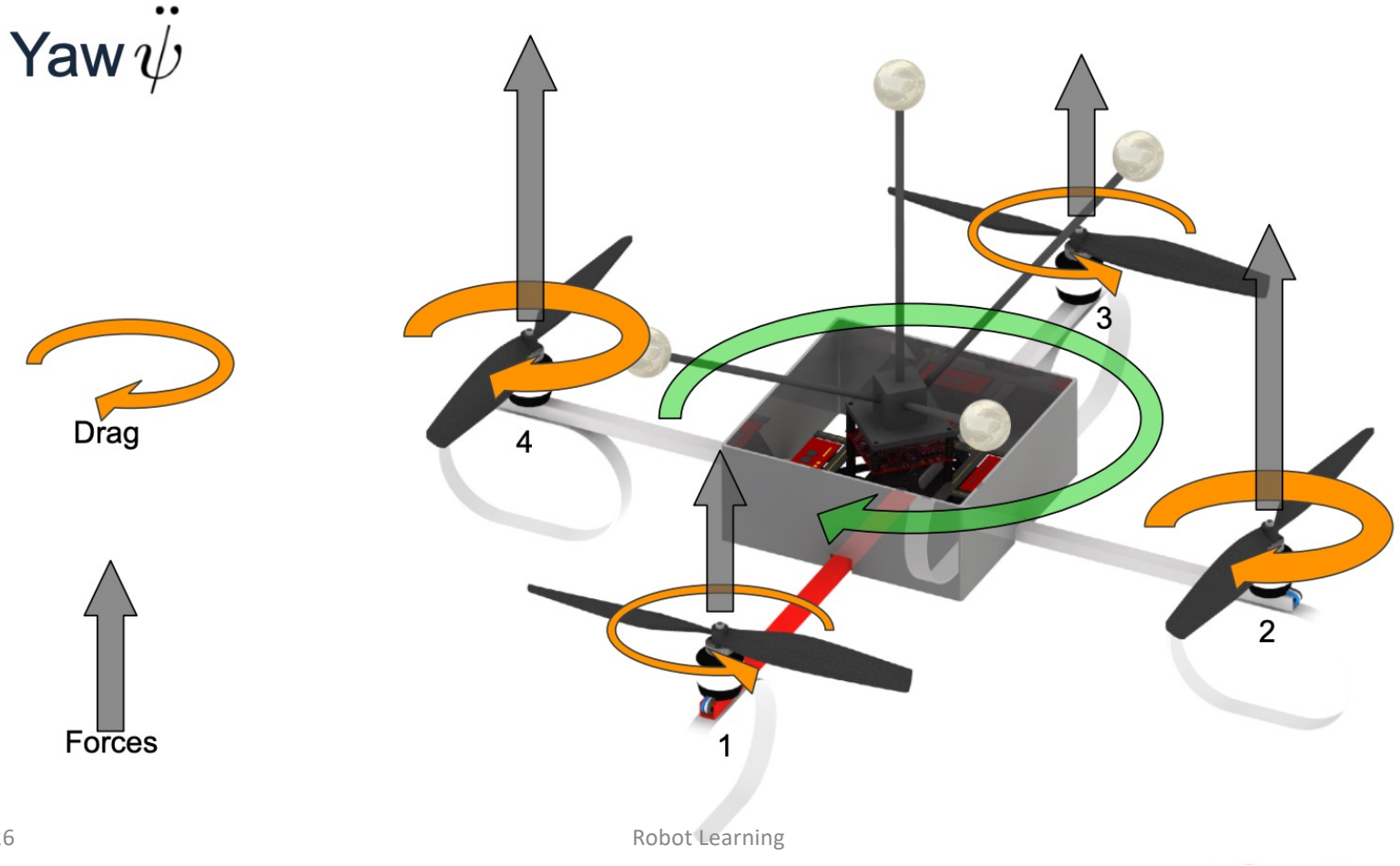
Quadrotor: working principle



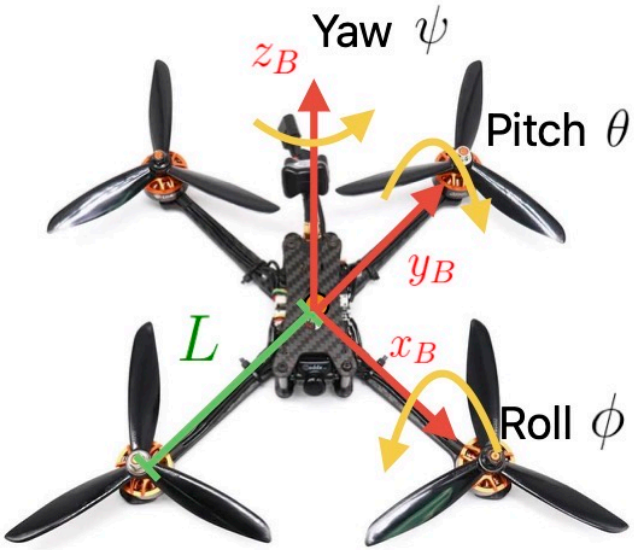
Quadrotor: working principle

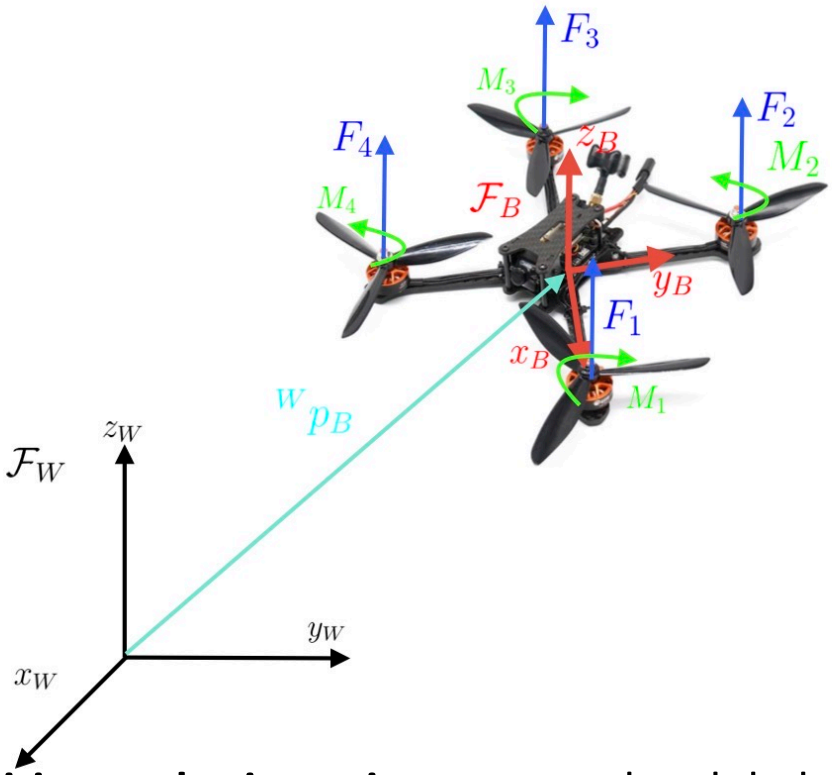


Quadrotor: working principle



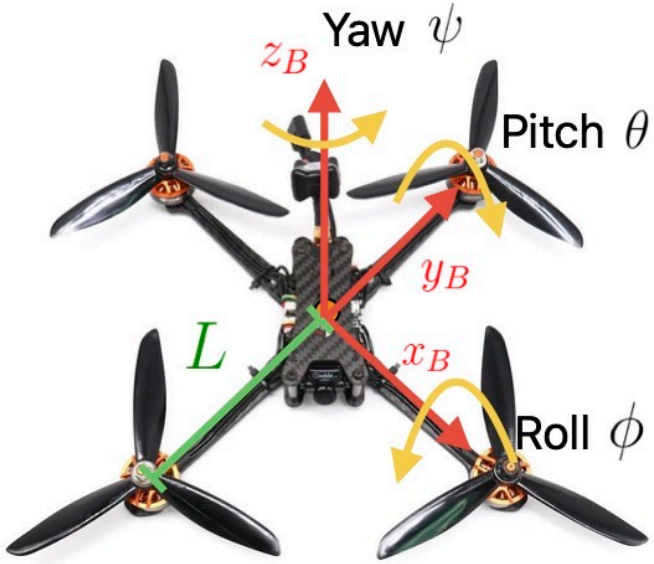
Quadrotor: frames and state





A quadrotor configuration in the world is defined by its **position and orientation** w.r.t. to the global frame (**6 variables**). Since the quadrotor only has **4 inputs** (the angular speed of the propellers) it is **underactuated**, so we specify trajectories in terms of **position** (3 variables) and **yaw** (1 variable).

Quadrotor: velocity mapping



The angular velocity of the quadrotor is expressed in the body frame

$$^{B}\omega = p x_B + q y_B + r z_B$$

The notation pqr is commonly used in aerial robotics. We can switch between roll-pitch-yaw rates and the body-frame angular velocity with a transformation

$$^{B}\boldsymbol{\omega} = \begin{bmatrix} p \\ q \\ r \end{bmatrix} = R_x(\phi)R_y(\theta)\begin{bmatrix} 0 \\ 0 \\ \dot{\psi} \end{bmatrix} + R_x(\phi)\begin{bmatrix} 0 \\ \dot{\theta} \\ 0 \end{bmatrix} + \begin{bmatrix} \dot{\phi} \\ 0 \\ 0 \end{bmatrix}$$

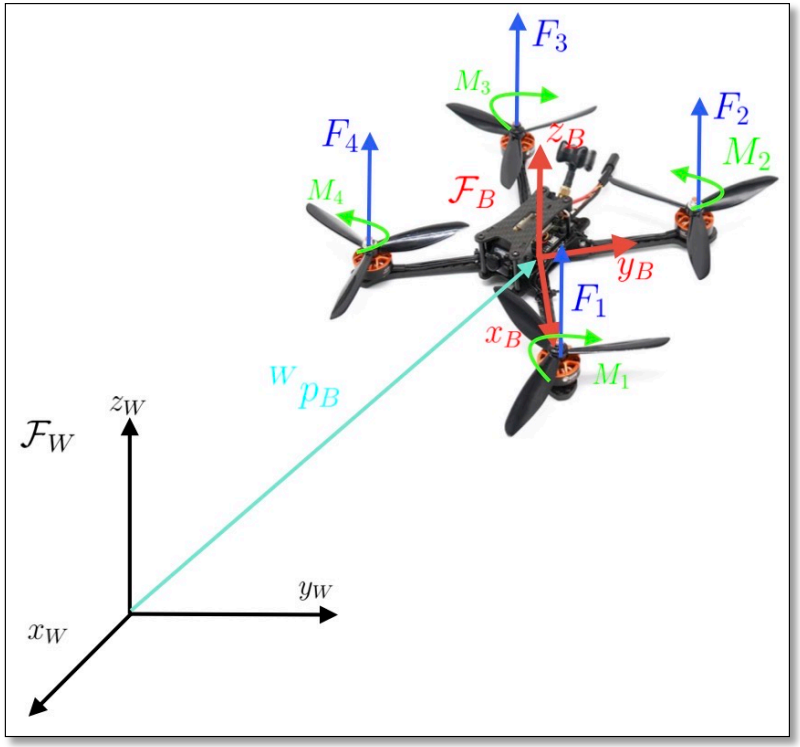
$$= \begin{bmatrix} 1 & 0 & -\sin(\theta) \\ 0 & \cos(\phi) & \sin(\phi)\cos(\theta) \\ 0 & -\sin(\phi) & \cos(\phi)\cos(\theta) \end{bmatrix} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}$$

Quadrotor: dynamics

Assumptions:

- Single rigid body
- The center of mass lies at the intersections of the 4 arms, so it is coincident with the origin of the body frame
- Motion in a low-speed and slow dynamics regimen

With these assumptions, using the **Newton-Euler** method is particularly simple



Equilibrium of forces and translational accelerations in world frame

$$m\,{}^W\ddot{\boldsymbol{p}}_B = \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} + {}^W\boldsymbol{f}$$

Equilibrium of torques and angular accelerations in body frame

$$I\,{}^B\dot{\omega} = -\,{}^B\omega \times \mathrm{I}\,{}^B\omega + {}^B\tau$$

Quadrotor: dynamics

What are the forces in the world frame?

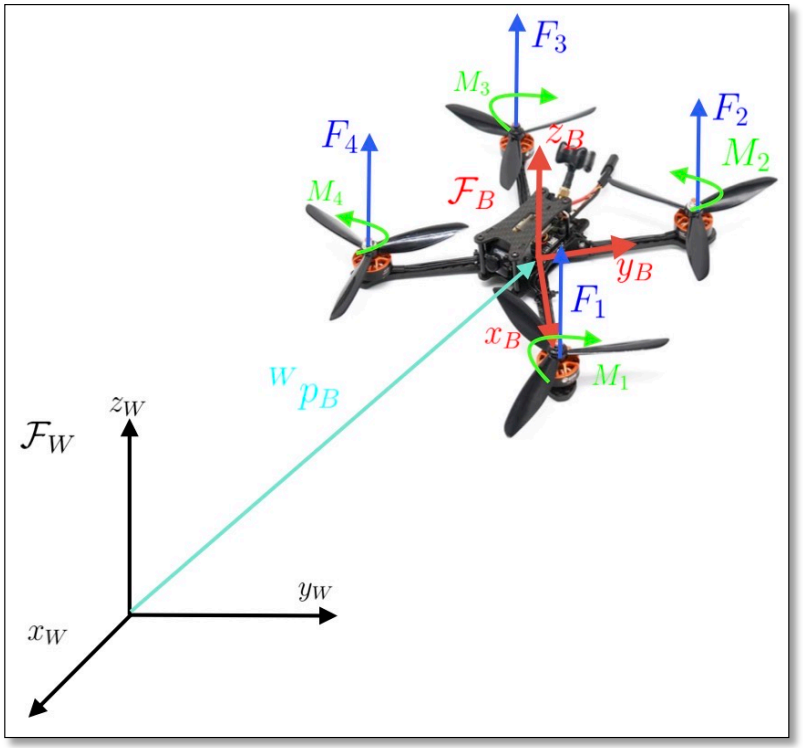
Each propeller produces a vertical force in the body frame, that is proportional to the squared velocity of the propeller. Therefore

$$\boldsymbol{f} = {}^{W}R_B(\phi,\theta,\psi)\begin{bmatrix}0\\0\\1\end{bmatrix}\left(F_1+F_2+F_3+F_4\right) \quad \text{with} \quad F_i = k_f\omega_i^2$$

What are the torques in the body frame?

Each propeller produces a torque vector with a vertical component, plus there is the momentum caused by the thrust forces that is proportional to the squared velocity of the propeller

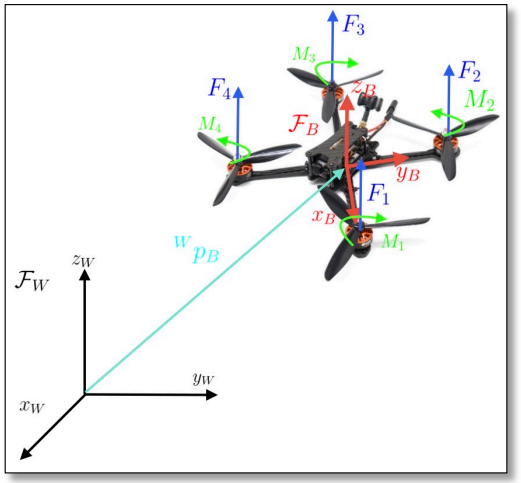
$${}^{B}\boldsymbol{\tau} = \begin{bmatrix} L(F_2-F_4)\\ L(F_3-F_1)\\ M_1-M_2+M_3-M_4\end{bmatrix} \quad \text{with} \quad M_i = k_M\omega_i^2$$



Quadrotor: dynamics

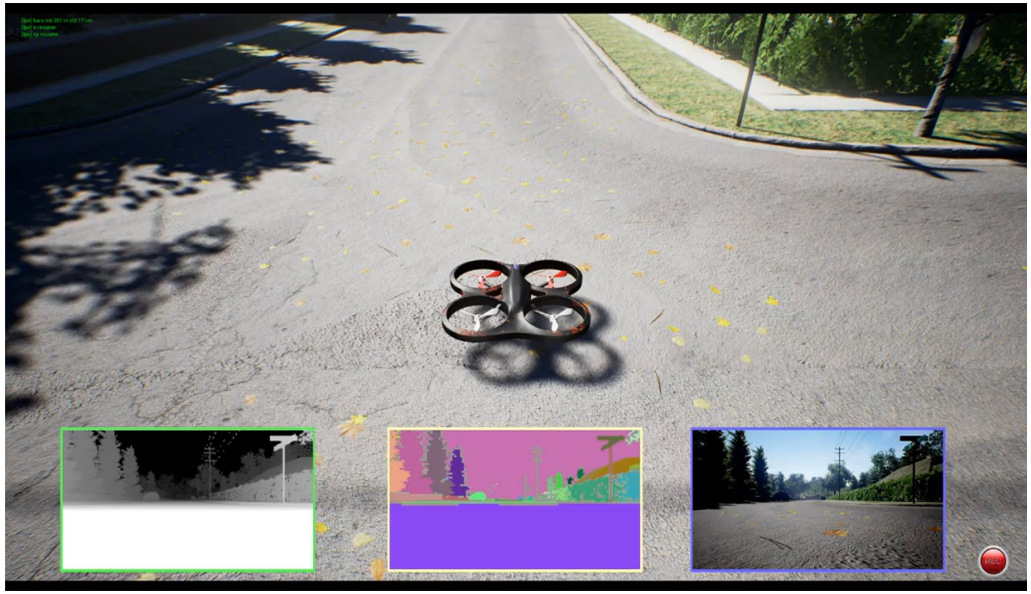
The final model is

$$\begin{bmatrix} mI_3 & \mathbf{0} \\ \mathbf{0} & I \end{bmatrix} \begin{bmatrix} {}^W\ddot{\boldsymbol{p}}_B \\ {}^B\dot{\boldsymbol{\omega}} \end{bmatrix} = \begin{bmatrix} -mg\boldsymbol{e}_3 \\ -{}^B\boldsymbol{\omega} \times I\, {}^B\boldsymbol{\omega} \end{bmatrix} + \begin{bmatrix} {}^W R_B(\phi,\theta,\psi) & \mathbf{0} \\ \mathbf{0} & I_3 \end{bmatrix} \begin{bmatrix} k_f & k_f & k_f & k_f \\ 0 & L\,k_f & 0 & -L\,k_f \\ -L\,k_f & 0 & L\,k_f & 0 \\ k_m & -k_m & k_m & -k_m \end{bmatrix} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix}$$



with $\boldsymbol{e}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$ $I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

Open-source simulators: Flightmare, Airsim



- Flightmare https://uzh-rpg.github.io/flightmare/
- AirSim https://microsoft.github.io/AirSim